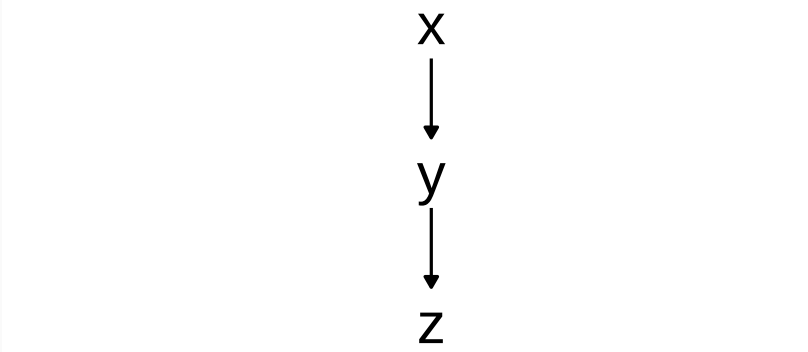
Bias and confounding with DAGs

Frank Edwards

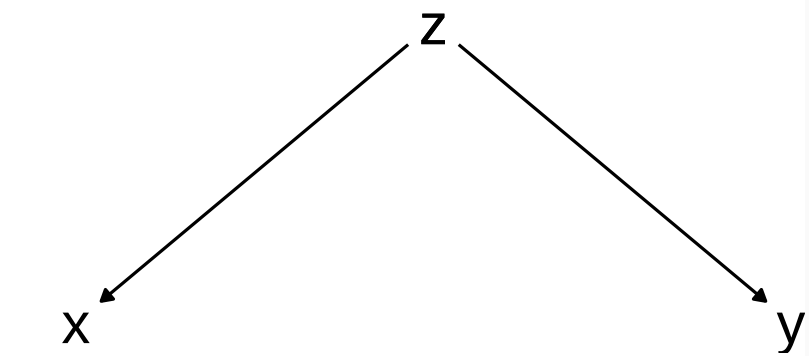
Four basic DAG structures

Subsections of DAGs are composed of these four structures

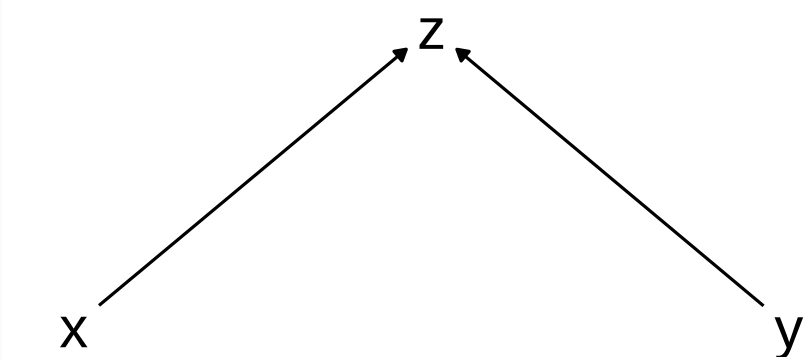
pipe



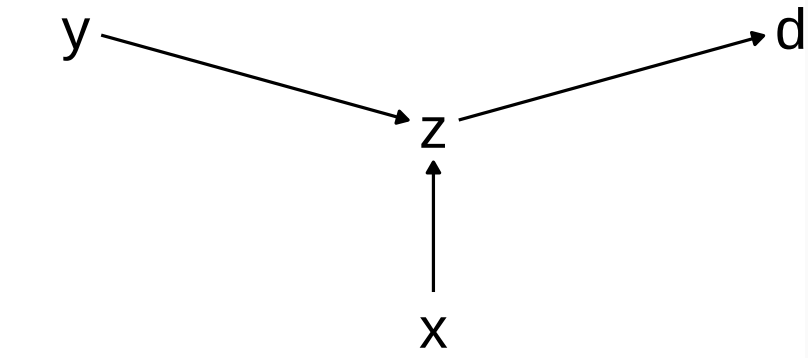
fork



collider



descendant



Basic features of the pipe

A pipe is a case of *mediation*. The effect of x on z is indirect, through y.



Mediation explained

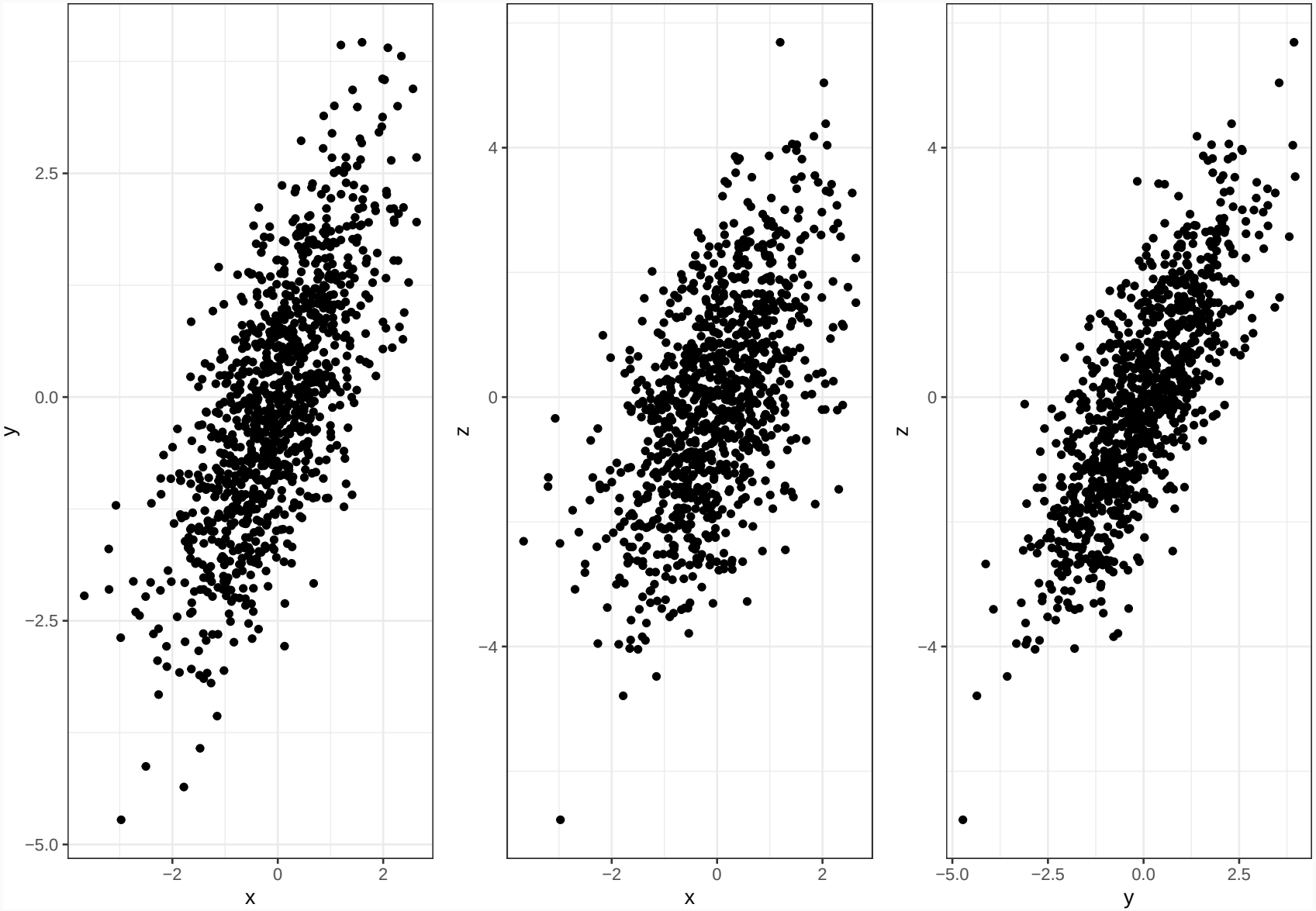
Complete mediation: the entire effect of x on z is accounted for by y



Let's simulate this to see how it works

```
x <- rnorm(1000, 0, 1)
y <- rnorm(1000, x, 1)
z <- rnorm(1000, y, 1)
```

Mediation simulation

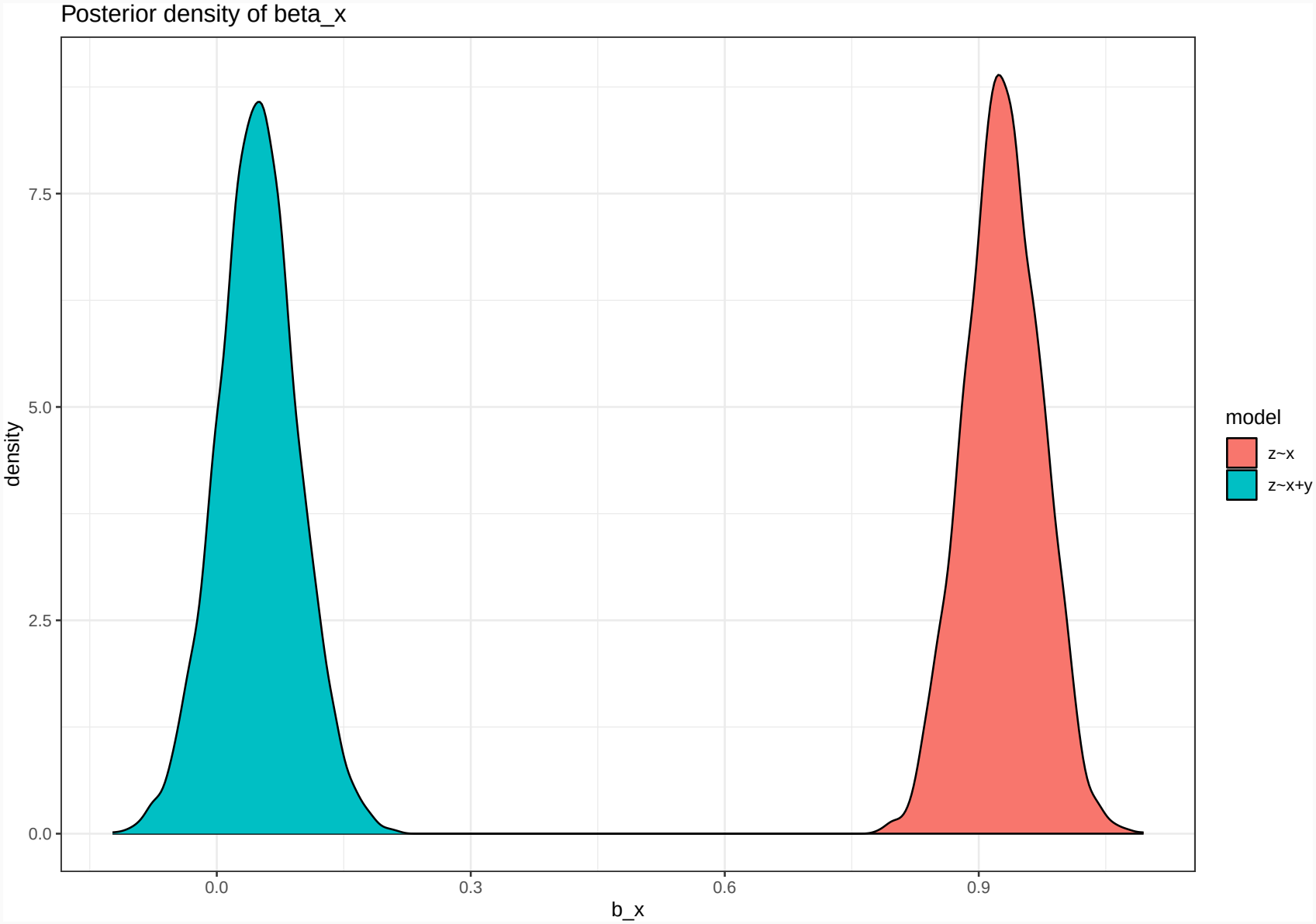


Let’s evaluate two models

I’ll use default (flat) priors for convenience. Generally not a good idea.

```
# Estimate model without conditioning on y
m_1 <- brm(z~x, data = pdat)
m_1_post <- as_draws_df(m_1) |>
  mutate(model = "z~x")
# Estimate model conditioning on y
m_2 <- brm(z~x+y, data = pdat)
m_2_post <- as_draws_df(m_2) |>
  mutate(model = "z~x+y")
```

Visualize


Posterior density of beta_x
density
7.5
5.0
2.5
0.0
0.0
0.3
0.6
0.9
b_x
model
z~x
z~x+y

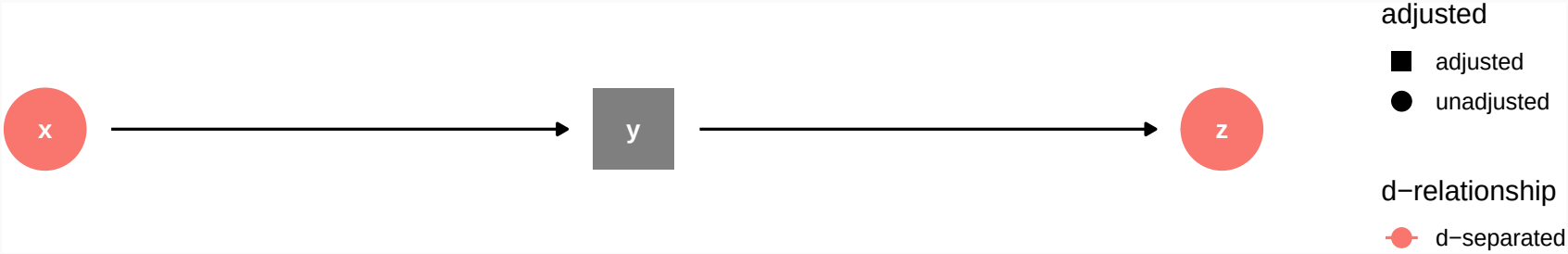
What this means in DAG terms: d-connection

The variables x and z are *d-connected* (direction)



What this means in DAG terms: d-separation

If we condition on y, the path $x \to y \to z$ is blocked, and x and z become *d-separated*



Mediation and post-treatment bias

Assume we are evaluating the effects of a CBT intervention t on externalizing behaviors b. A colleague suggest we should control for depression d in our analysis.

Assume this DAG represents the process



Simulation

```r
# baseline externalizing behaviors
b0 <- rnorm(1000, 0, 1)
# treatment assignment
t <- rbinom(1000, 1, 0.5)
# depression (affected by t)
d <- rnorm(1000, -t, 1)
# b1 affected by d and b0
b1 <- rnorm(1000, b0+d, 1)
e1_dat <- data.frame(b0, t, d, b1)
```

```r
m_e1 <- brm(b1 ~ b0 + d + t, data = e1_dat)
```

Let’s see what happens when we follow the well-intentioned advice

```
m_e1 <- brm(b1 ~ b0 + d + t, data = e1_dat)
```

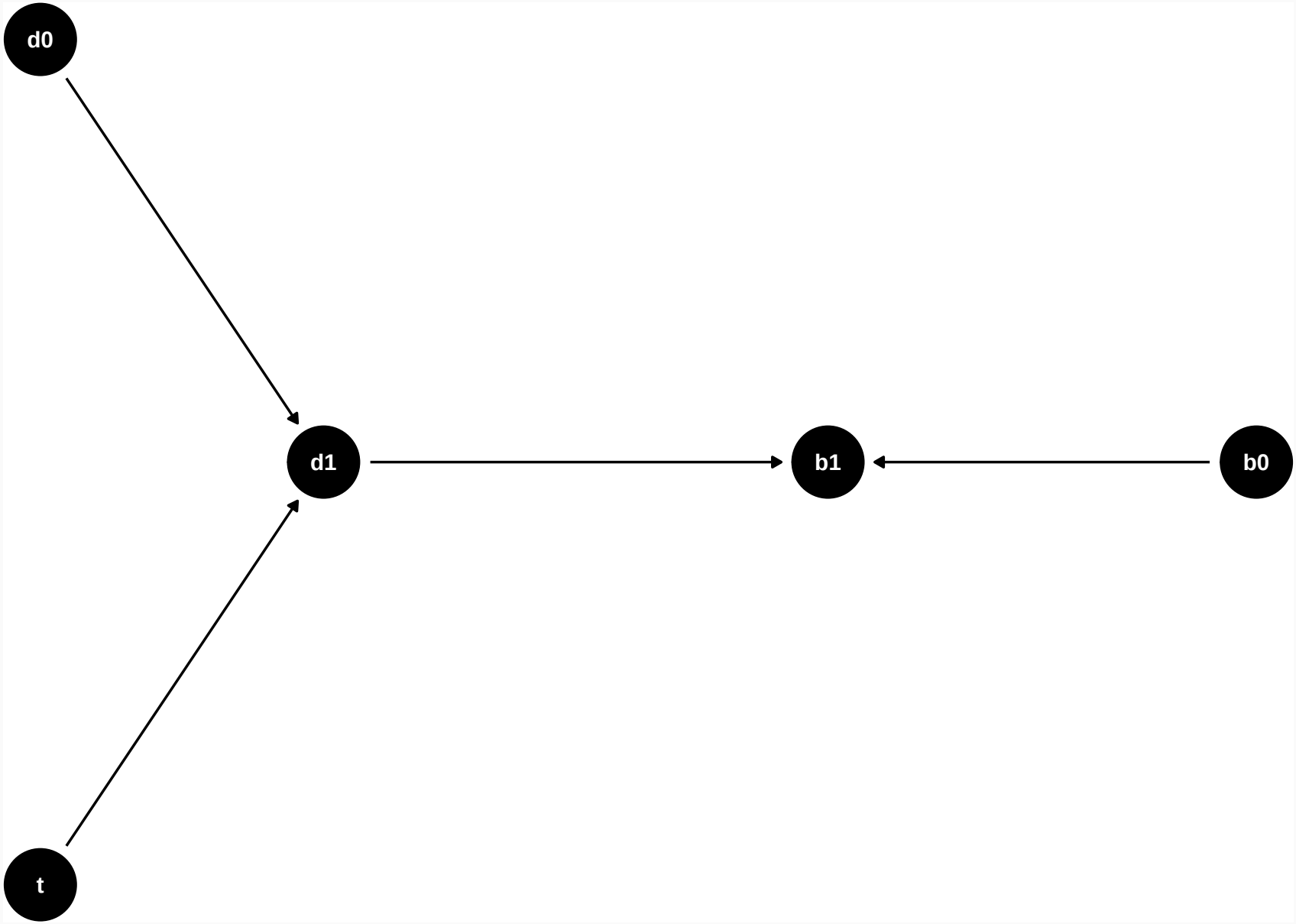
```
            Estimate  Est.Error        Q2.5     Q97.5
Intercept 0.01812248 0.04546498 -0.07147565 0.1063481
b0        0.99316527 0.03293091  0.92984390 1.0573068
d         1.05065847 0.03093234  0.98908445 1.1107998
t         0.07946302 0.06968562 -0.05597495 0.2190705
```

And when we don’t

```
m_e1_2 <- brm(b1 ~ b0 + t, data = e1_dat)
```

```
              Estimate  Est.Error       Q2.5      Q97.5
Intercept  0.009875143 0.06501888 -0.1178881  0.1380826
b0         1.013348978 0.04814096  0.9199519  1.1108172
t         -0.927310403 0.09178114 -1.1062091 -0.7464213
```

A more realistic example



Simulating

```r
n_sims <- 1000
# baseline externalizing behaviors
b0 <- rnorm(n_sims, 0, 1)
# baseline depression
d0 <- rnorm(n_sims)
# treatment assignment
t <- rbinom(n_sims, 1, 0.5)
# depression t1 (affected by t)
d1 <- rnorm(n_sims, d0 - t, 1)
# b1 affected by d1 and b0
b1 <- rnorm(n_sims, b0+d1, 1)
edat <- data.frame(b0, d0, t, d1, b1)
```

Fitting the models

```
# fit model with pre-treatment depression
m_e1_2_1 <- brm(b1~b0+d0+t, data = edat)
# with both
m_e1_2_2 <- brm(b1~b0+d0+d1+t, data = edat)
# and without any
m_e1_2_3 <- brm(b1~b0+t, data = edat)
```

And compare posteriors

```
fixef(m_e1_2_1)
```

```
             Estimate  Est.Error        Q2.5      Q97.5
Intercept  0.04419133 0.06647433 -0.08931175  0.1717201
b0         0.91536490 0.04396741  0.82977207  1.0008289
d0         1.07324465 0.04351380  0.98771163  1.1577826
t         -1.08993773 0.08967754 -1.26565807 -0.9114954
```

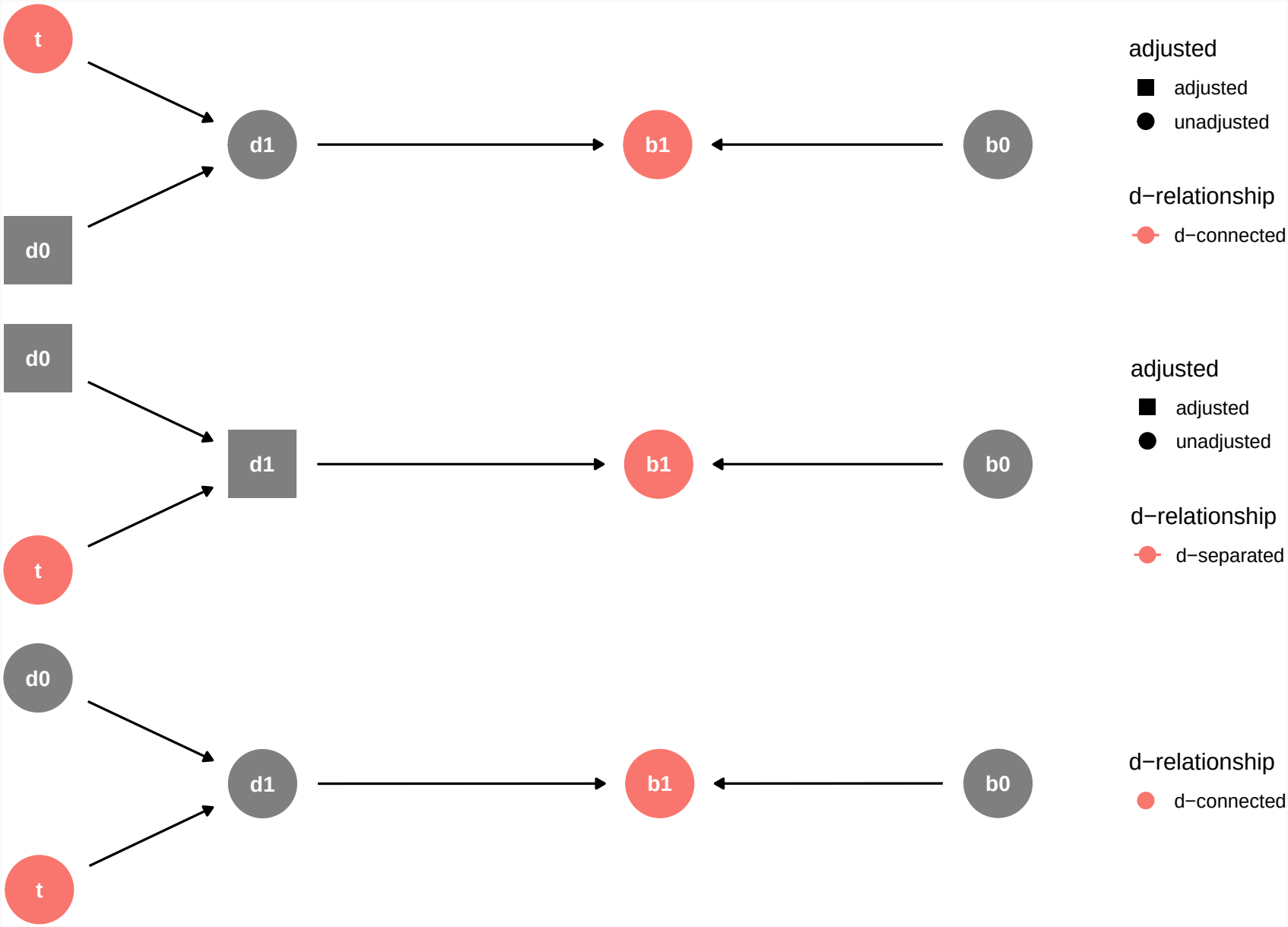
```
fixef(m_e1_2_2)
```

```
             Estimate  Est.Error        Q2.5     Q97.5
Intercept  0.03623533 0.04199151 -0.04643520 0.1181129
b0         0.94867092 0.03081919  0.88882588 1.0077665
d0         0.03361181 0.04144042 -0.04610153 0.1138285
d1         0.98975730 0.02799761  0.93264647 1.0434403
t         -0.07064310 0.06616589 -0.19894822 0.0537560
```

```
fixef(m_e1_2_3)
```

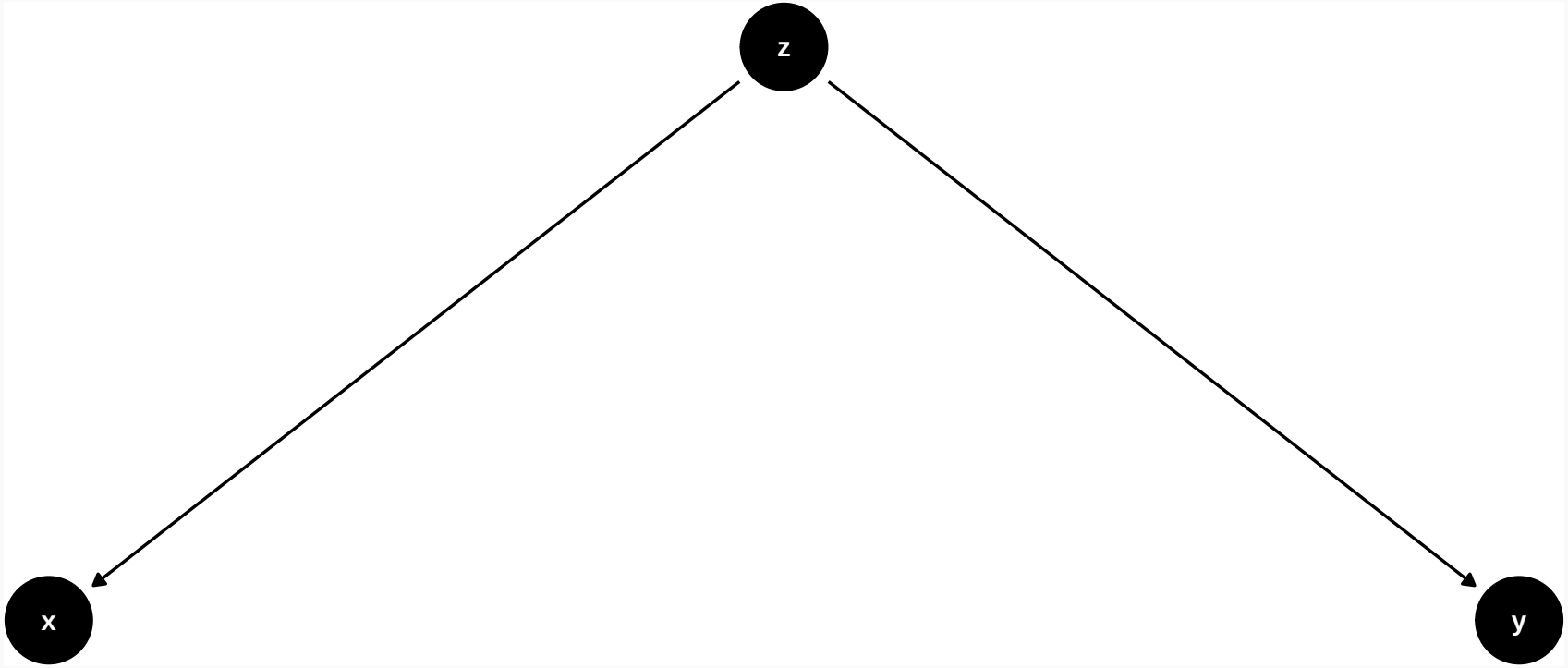
```
             Estimate  Est.Error       Q2.5      Q97.5
Intercept  0.04491146 0.08186312 -0.1187572  0.2080179
b0         0.88004735 0.05613771  0.7706247  0.9903133
t         -0.99131200 0.11405532 -1.2137390 -0.7701098
```

What's happening (DAG version)



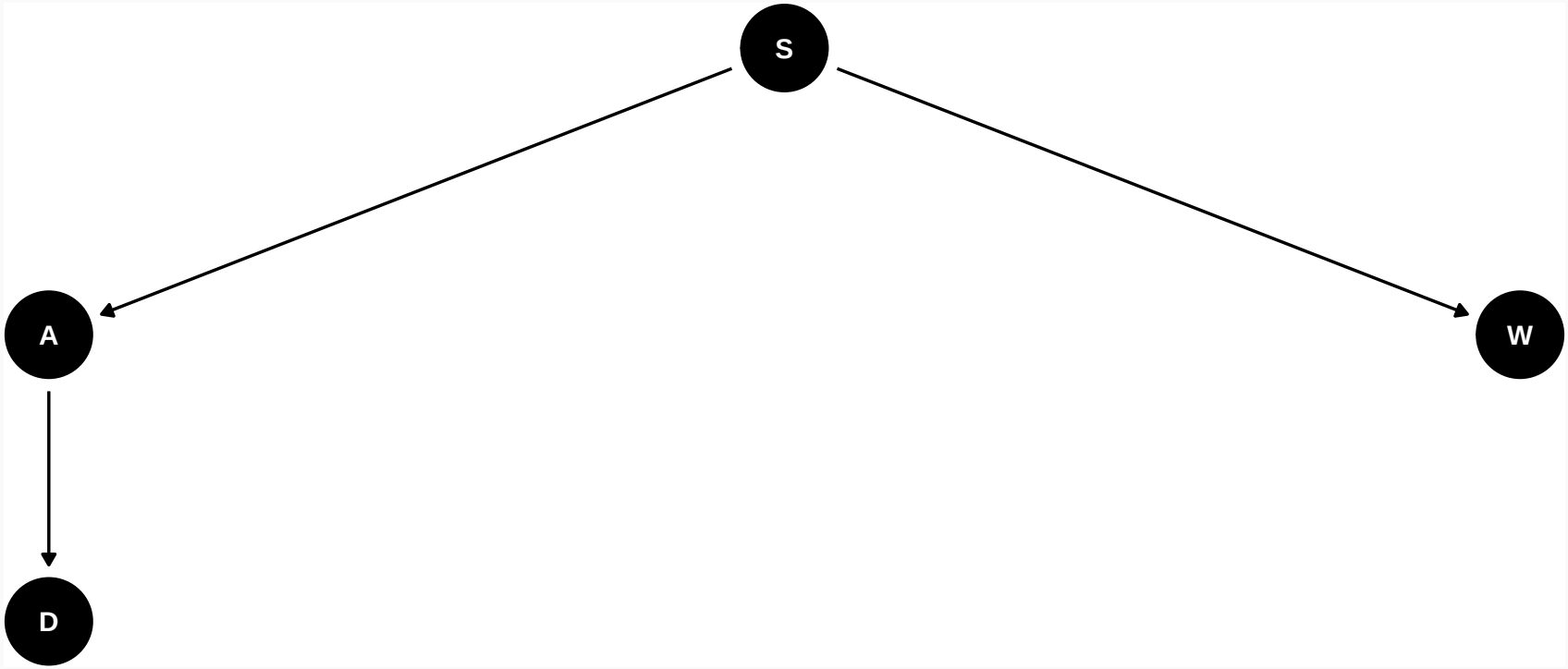
Forks

A fork is the classic lemonade sales and homicide rates confounder: a common cause.



An example fork

Waffle House and Divorce

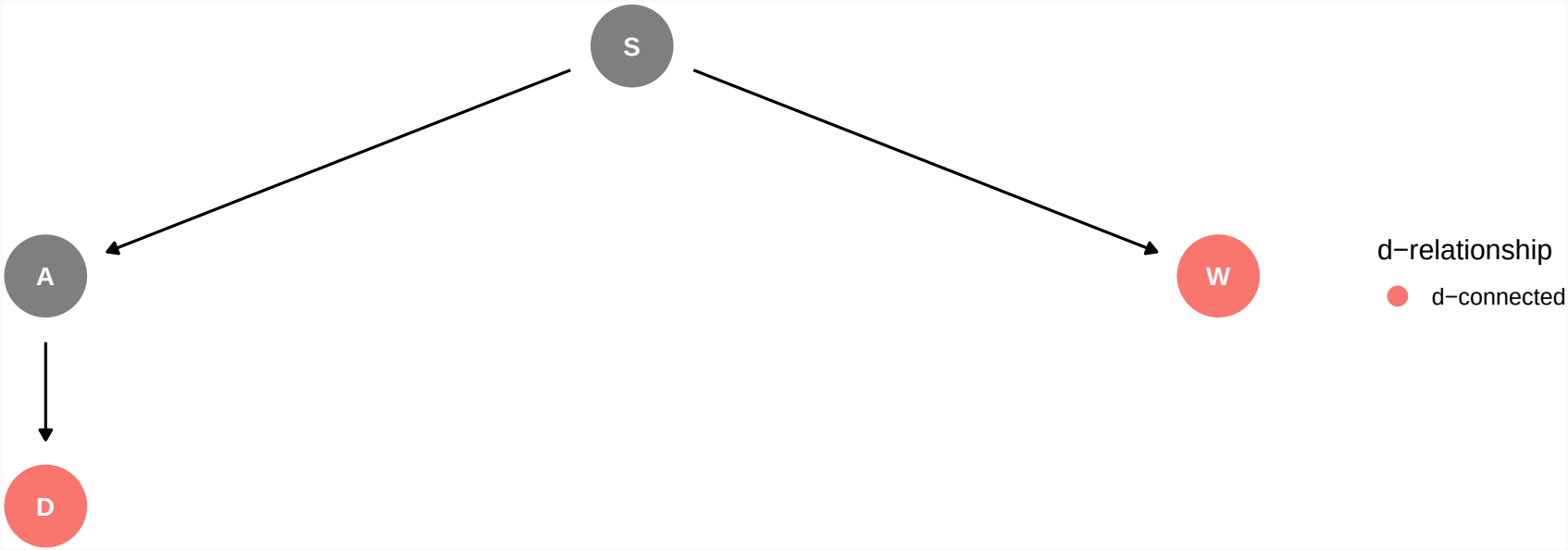


D-separation in a fork

```
adjustmentSets(waffle_dag)
```

```
{ A }
{ S }
```

```
ggdag_dseparated(waffle_dag, from = "W")
```



In action

We'll prefer an adjustment for the variable closest to the outcome in the DAG for efficiency (reduced noise).

```
data("WaffleDivorce")
mw1 <- brm(scale(Divorce) ~ scale(WaffleHouses/Population),
           data = WaffleDivorce)
mw2 <- brm(scale(Divorce) ~ scale(WaffleHouses/Population) +
             scale(MedianAgeMarriage),
           data = WaffleDivorce)
mw3 <- brm(scale(Divorce) ~ scale(WaffleHouses/Population) +
             factor(South),
           data = WaffleDivorce)
mw4 <- brm(scale(Divorce) ~ scale(WaffleHouses/Population) +
             scale(MedianAgeMarriage) + factor(South),
           data = WaffleDivorce)
```

In action: results

If our DAG is correct, we expect to see a $W \not\perp D$ in mw1, $W \perp D|A$ in mw2, $W \perp D|S$ in mw3, and $W \perp D|S, A$ in mw 4. Note the impacts of collinearity

```
                              Estimate Est.Error       Q2.5     Q97.5
Intercept                   -0.003585286 0.1337108 -0.2666989 0.2617870
scaleWaffleHousesDPopulation  0.366788240 0.1334284  0.1022975 0.6286968

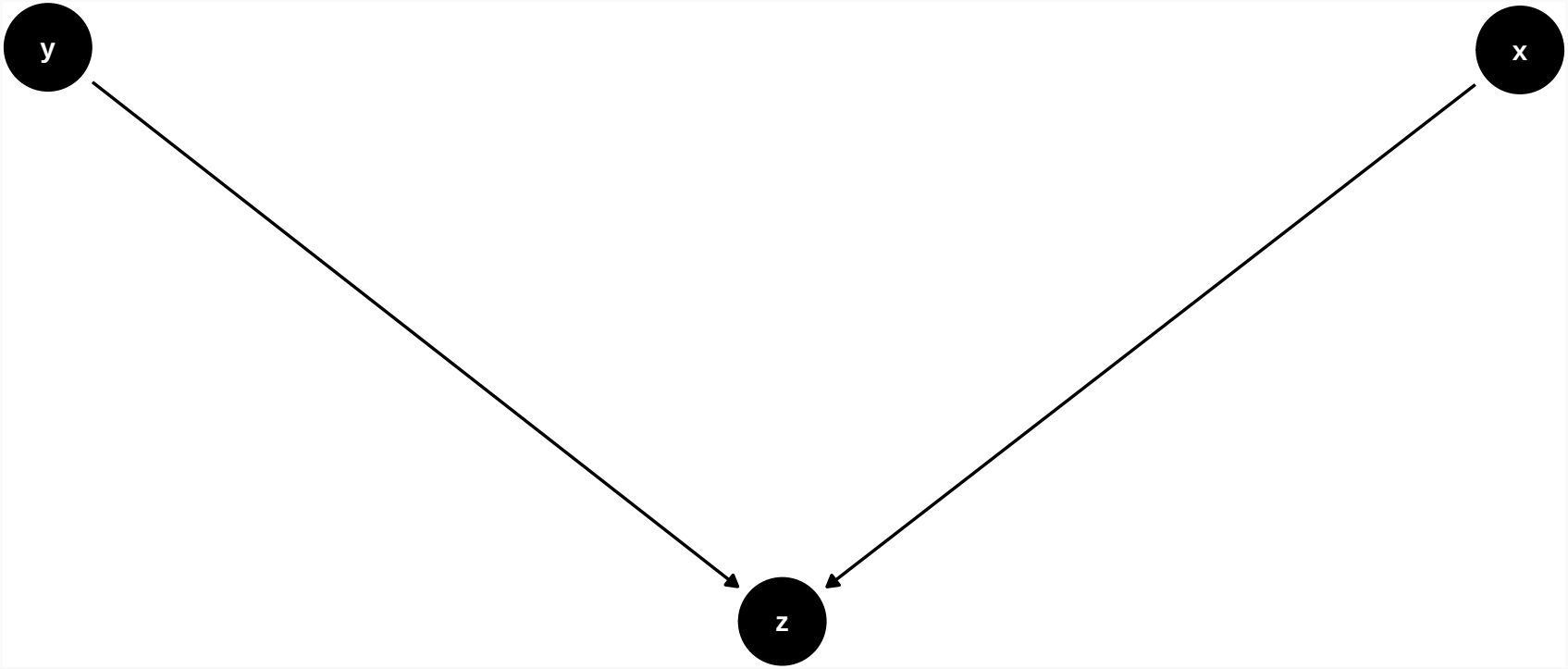
                              Estimate Est.Error        Q2.5      Q97.5
Intercept                   0.001217924 0.1151967 -0.23136942  0.2278545
scaleWaffleHousesDPopulation  0.269513803 0.1143221  0.04827007  0.4934025
scaleMedianAgeMarriage       -0.547494216 0.1167721 -0.77920491 -0.3146272

                               Estimate Est.Error       Q2.5     Q97.5
Intercept                   -0.1019066 0.1860592 -0.4706083 0.2656788
scaleWaffleHousesDPopulation  0.2439298 0.2057375 -0.1555834 0.6560912
factorSouth1                  0.3509804 0.4553627 -0.5376252 1.2222359

                              Estimate Est.Error        Q2.5      Q97.5
Intercept                   -0.009828219 0.1615558 -0.32717454  0.3058288
scaleWaffleHousesDPopulation  0.261075446 0.1789414 -0.09473722  0.6123868
scaleMedianAgeMarriage       -0.546626822 0.1202580 -0.78577178 -0.3037291
factorSouth1                  0.029327336 0.3990697 -0.75297197  0.8297952
```

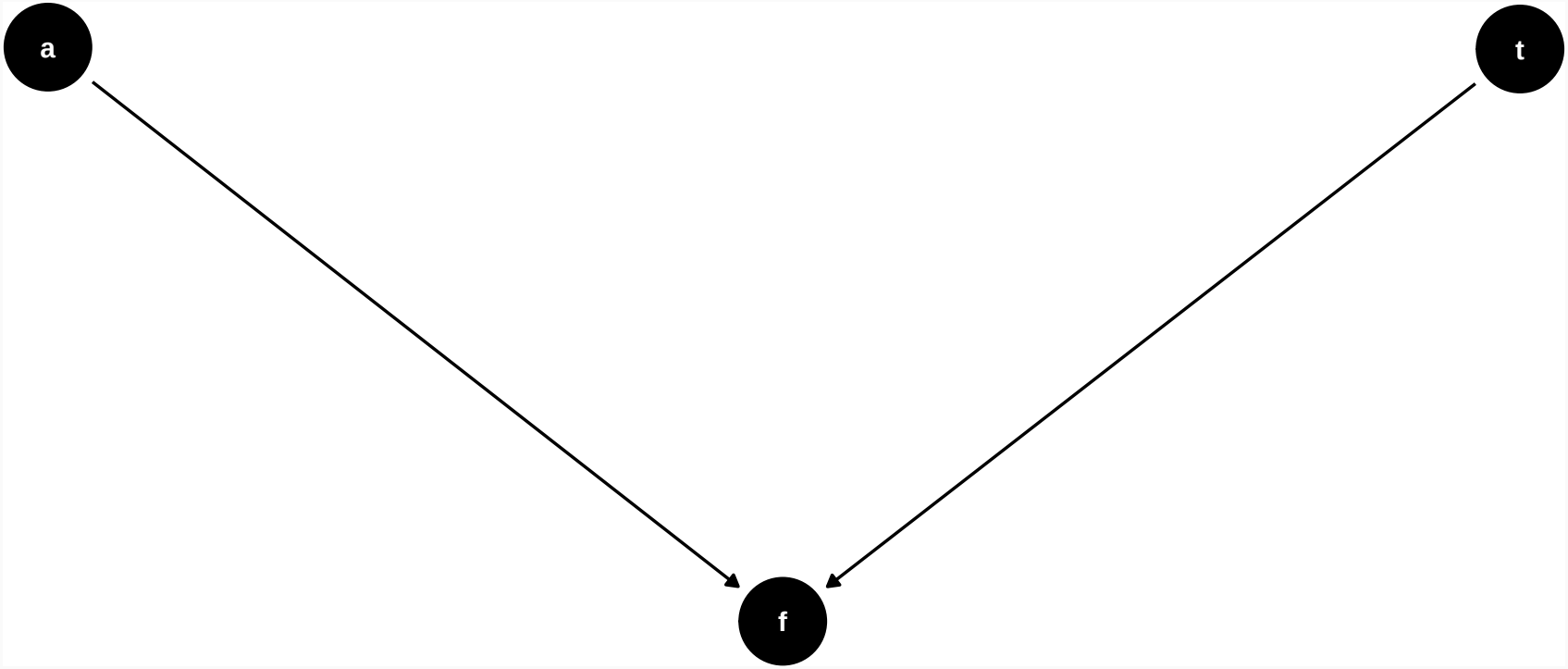
The structure of a collider

A *collider* is a node that has two arrows pointing into it. A collider *blocks* the flow of information between x and y, unless we condition on it. The impact of colliders is similar to *selection bias*.



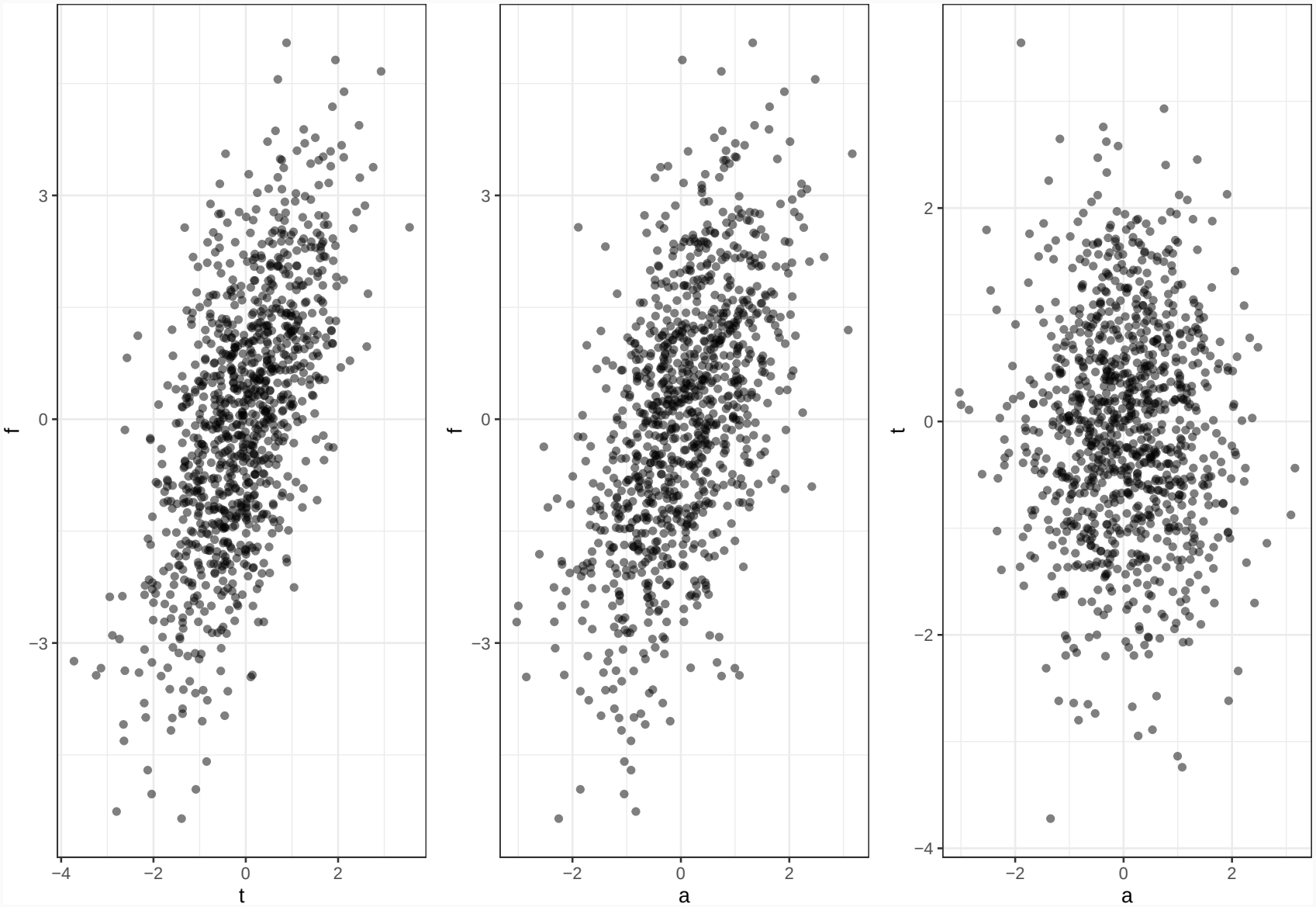
A toy example

Movie stars become famous for being conventionally attractive and/or being a good actor. Conventional attractiveness and acting talent are not associated with each other.

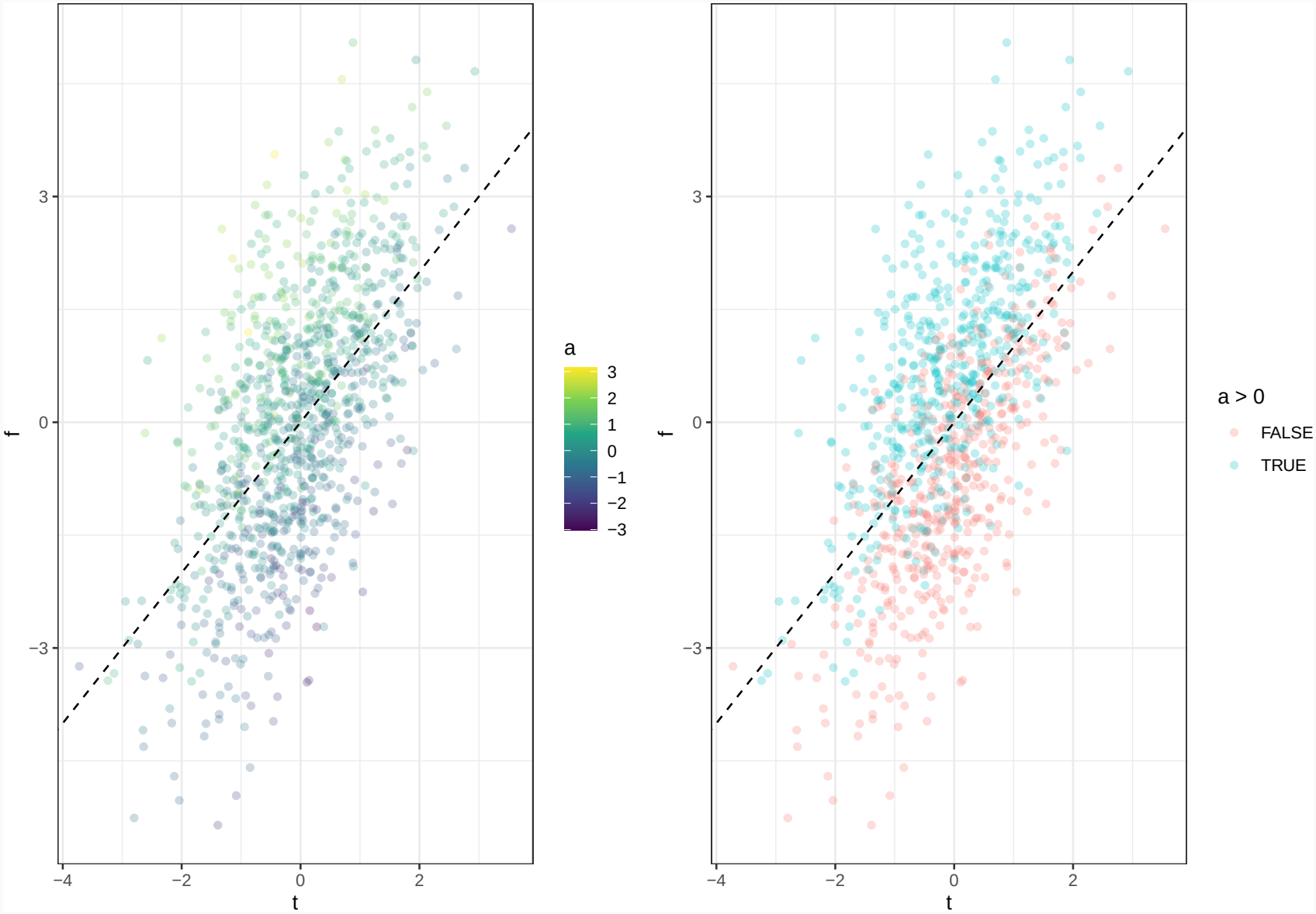


Simulate our example

Visualize



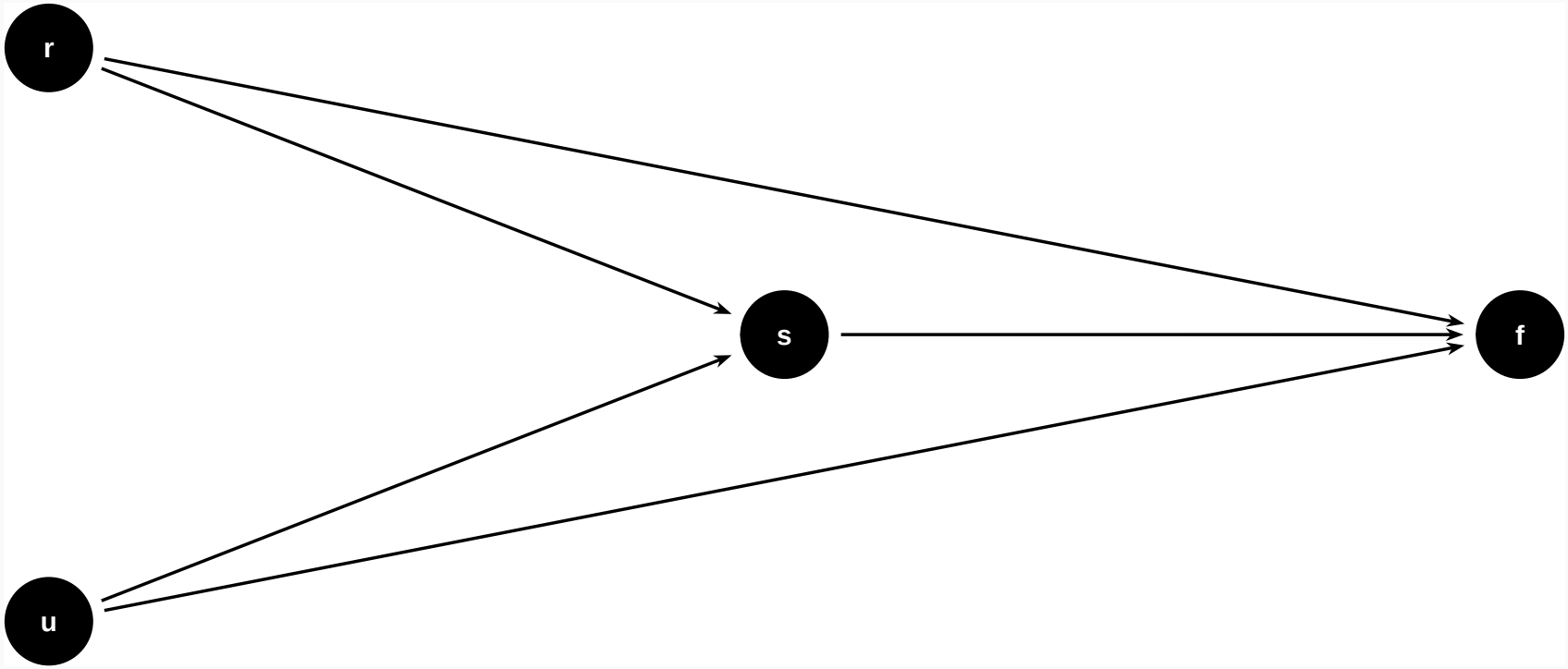
The impact of the selection process



A more serious example: racism and police use of force

We want to estimate the effect of race r on police use of force f

r: perceived race, u: officer perceptions of suspiciousness or danger, s: stop, f: force

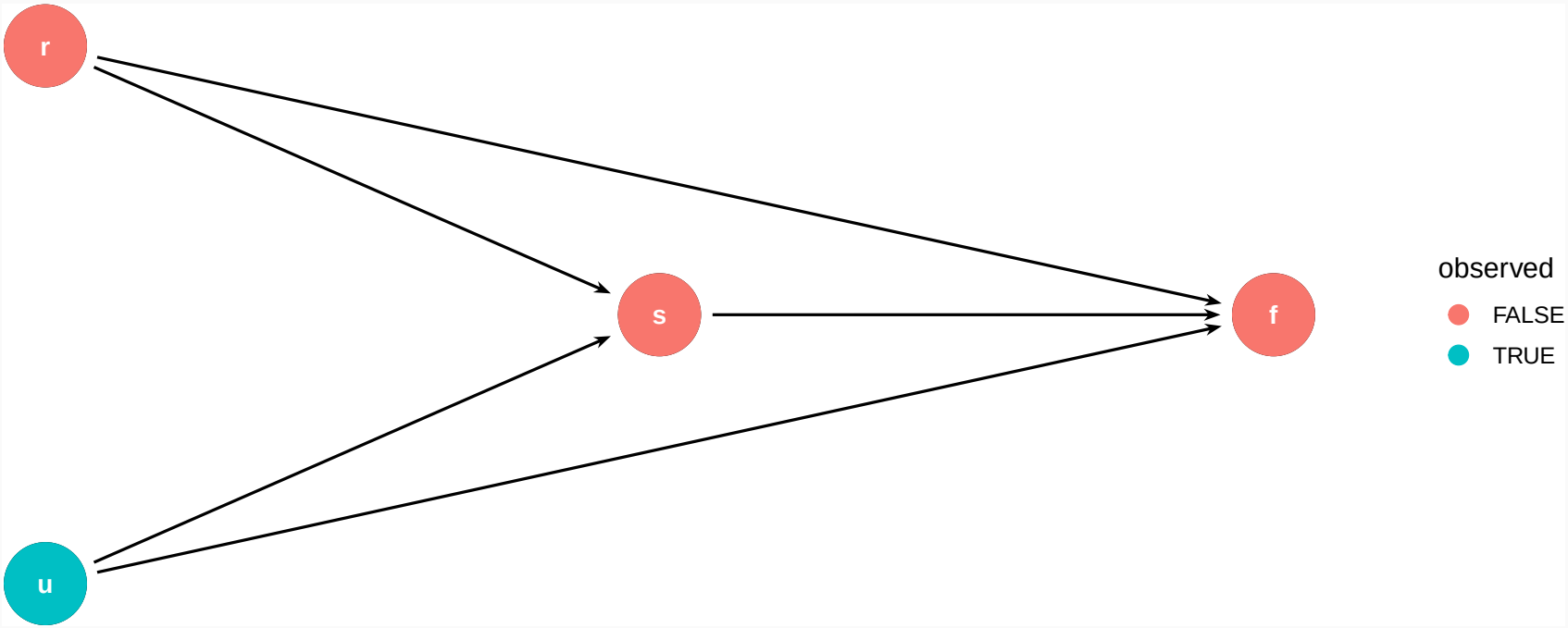


{for much more on this, see Knox, Dean, Will Lowe, and Jonathan Mummolo. 2020. "Administrative Records Mask Racially Biased Policing." American Political Science Review 114 (3): 619–37. DOI: 10.1017/S0003055420000039.}

A more serious example: racism and police use of force

The reality

r: perceived race, u: officer perceptions of suspiciousness or danger, s: stop, f: force



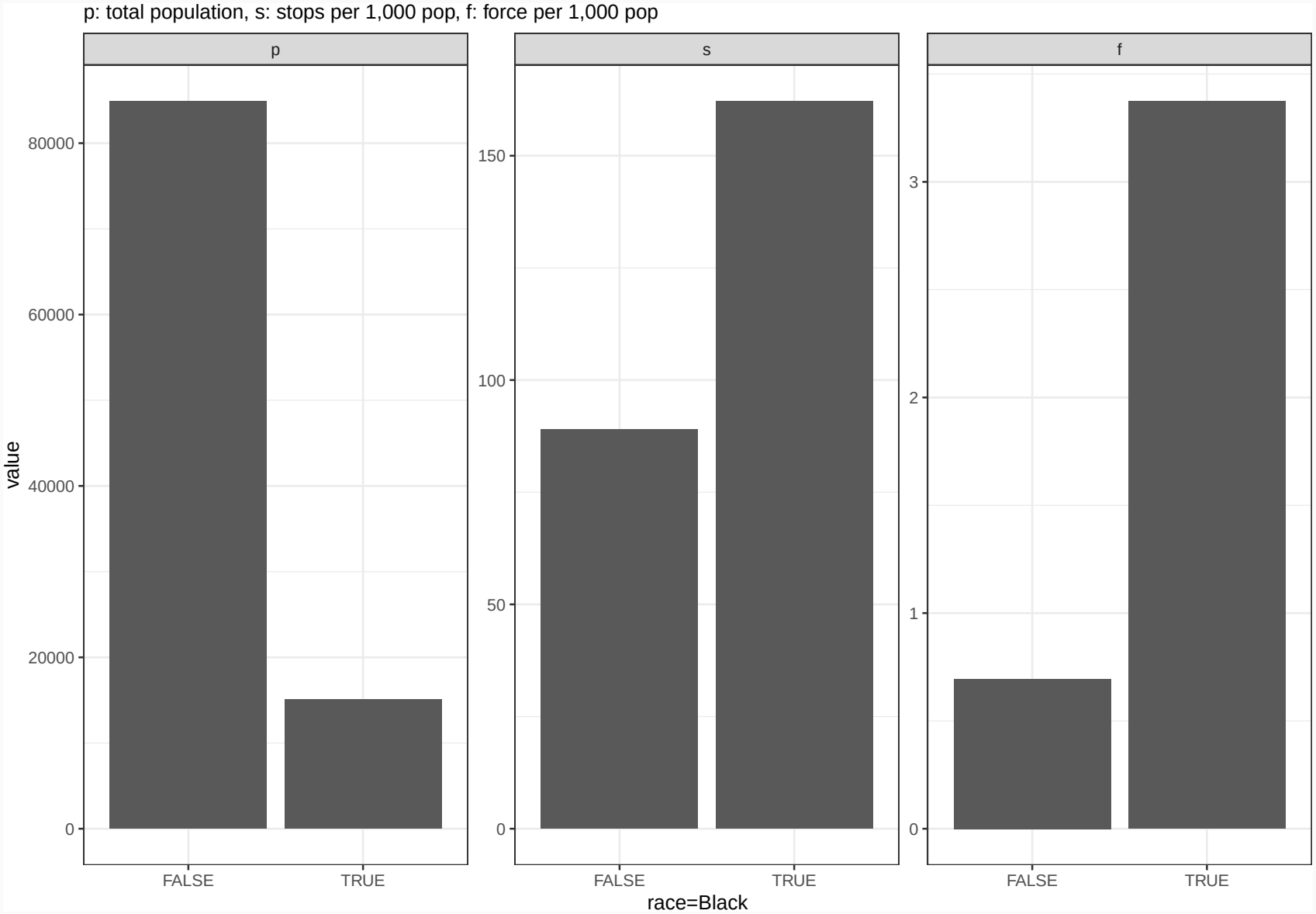
{Knox, Dean, Will Lowe, and Jonathan Mummolo. 2020. "Administrative Records Mask Racially Biased Policing." American Political Science Review 114 (3): 619–37. DOI: 10.1017/S0003055420000039.}

Simulating the problem

Assumptions: Empirical estimates suggest about a 1 in 10 rate of police traffic stops in the population and a 1 in 1000 rate of use of force.

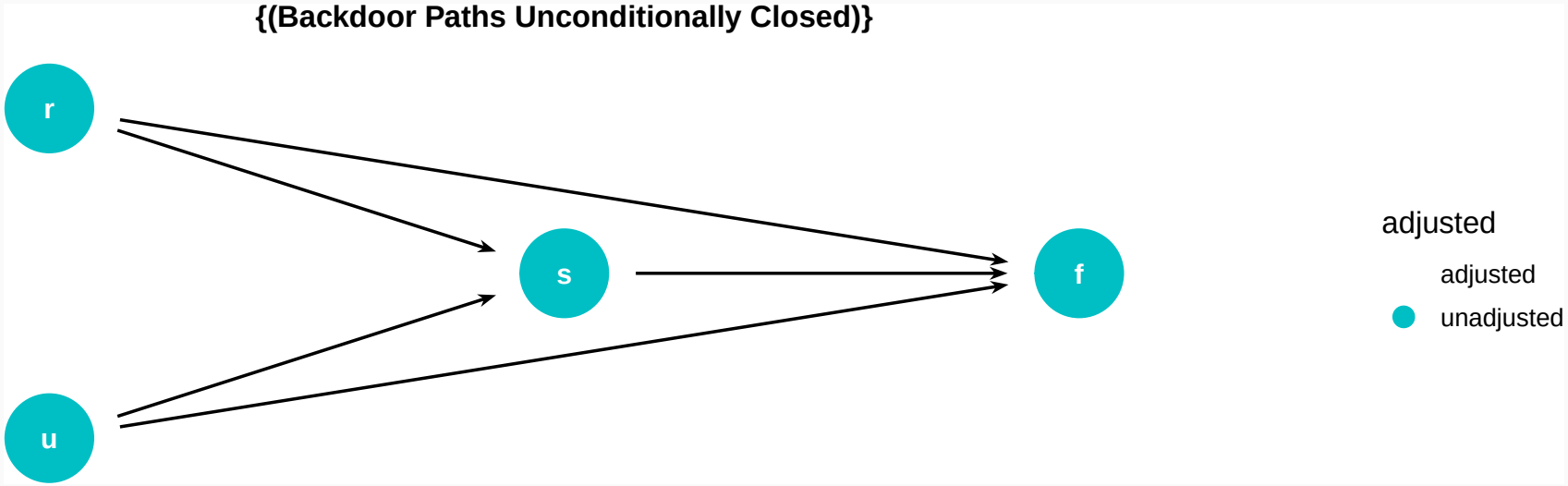
```
n_sims <- 1e5
# simulate race, 1 = black, US proportion is ~ 0.15
r <- rbinom(n_sims, 1, 0.15)>0
# simulate police suspicion with r->u
u <- rnorm(n_sims, 0, 2)
# simulate stop propensity with u->s, r->s
s <- rnorm(n_sims, u + r, 2)
# dichotomize with top decile as stops
s <- s>quantile(s, 0.9)
# simulate as propensity  u->f,
f <- rnorm(n_sims, u + r, 1)
# condition on stop, then subset to top 1% of propensity
f <- f>=quantile(f[s==1], 0.99)
pol_sim_dat <- data.frame(r, s, f)
```

Visualizing the simulated data



Estimating a causal effect of race

The collider s blocks the backdoor path without adjustment.

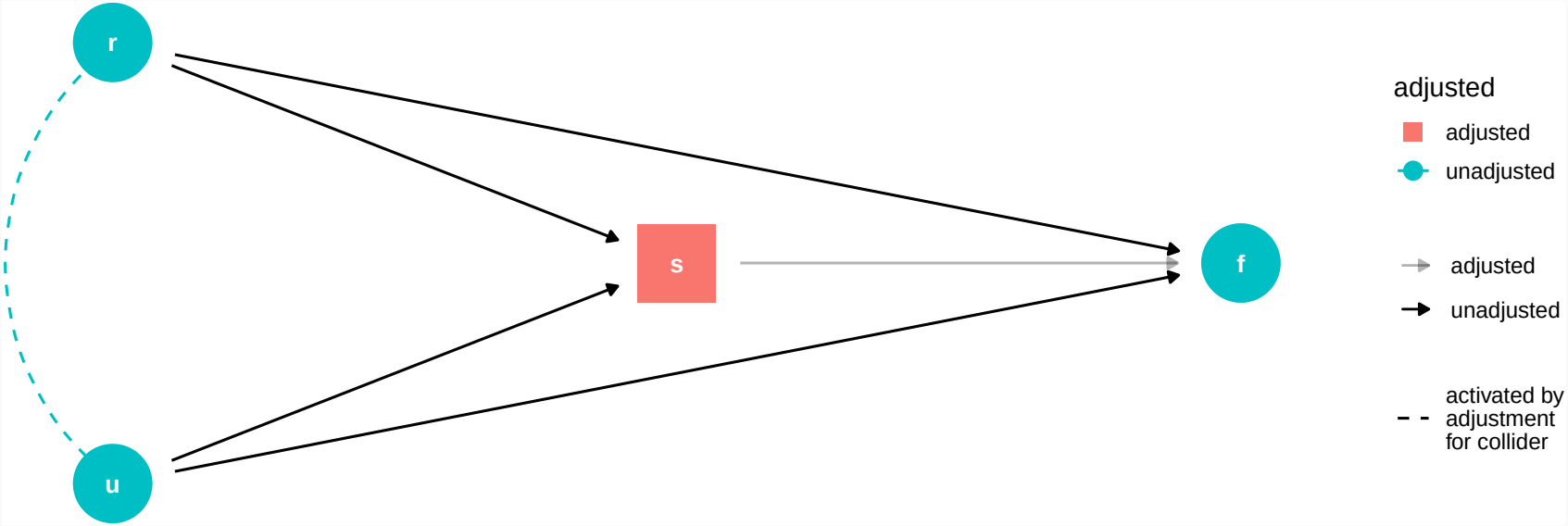


Fitting the model

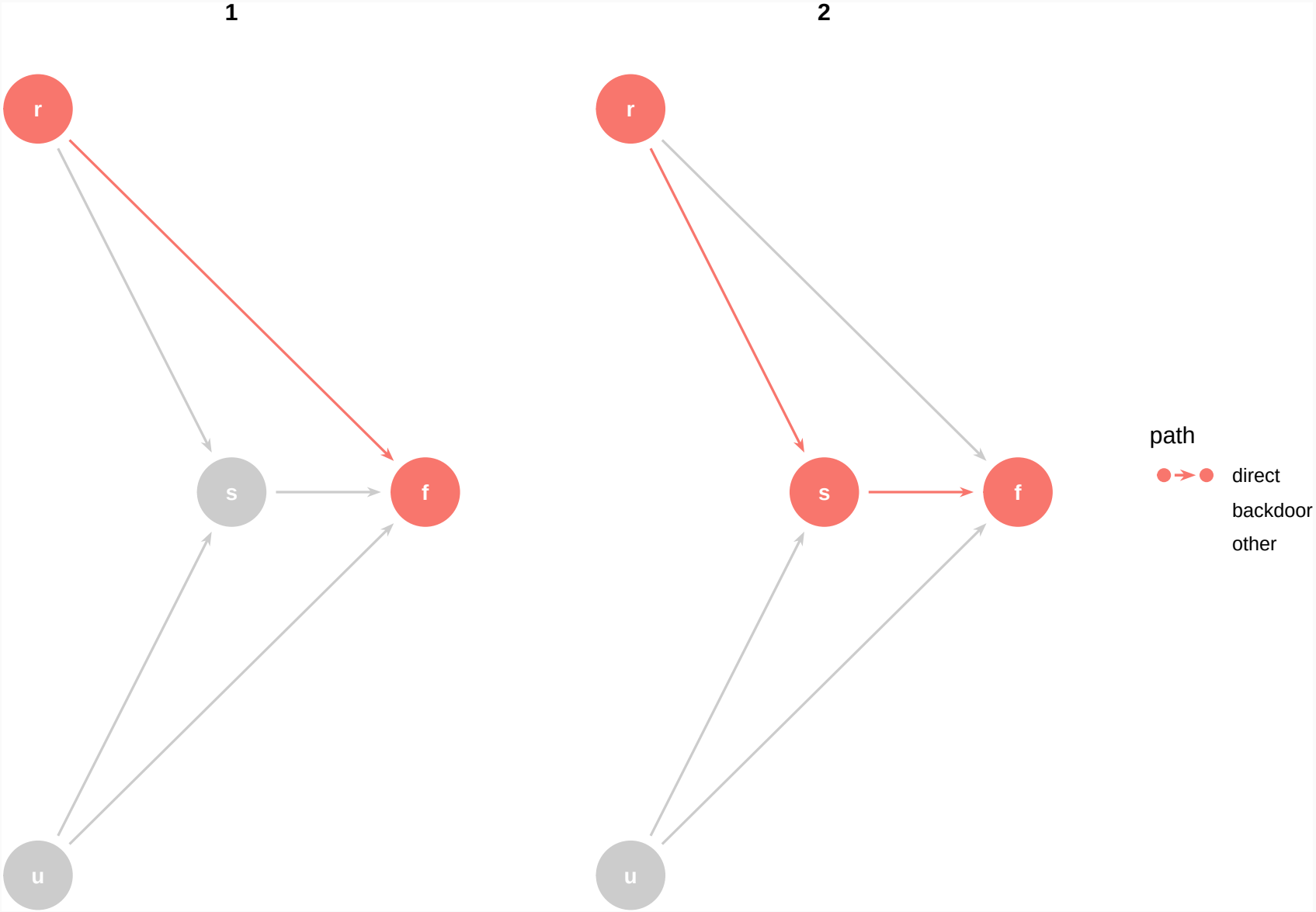
```
# estimate logit model
m_pol0 <- brm(f~r, data = pol_sim_dat, family = "bernoulli")
```

The problem: conditioning on a collider

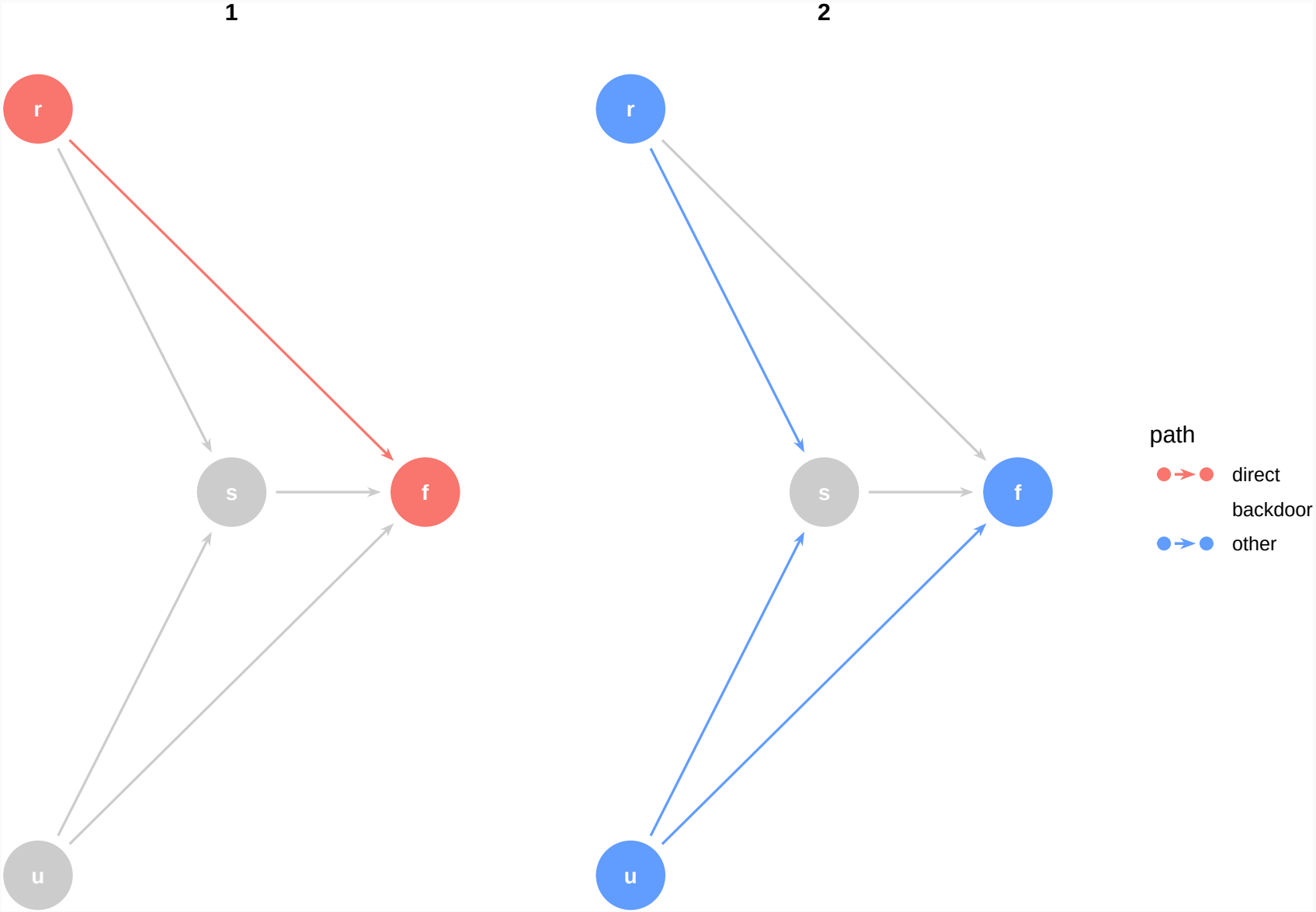
Most studies of police violence use administrative data for estimation, which are generally *only* collected after a stop. These studies are *conditioning* on s.



What's happening here: no conditioning



What's happening here: conditioning on s



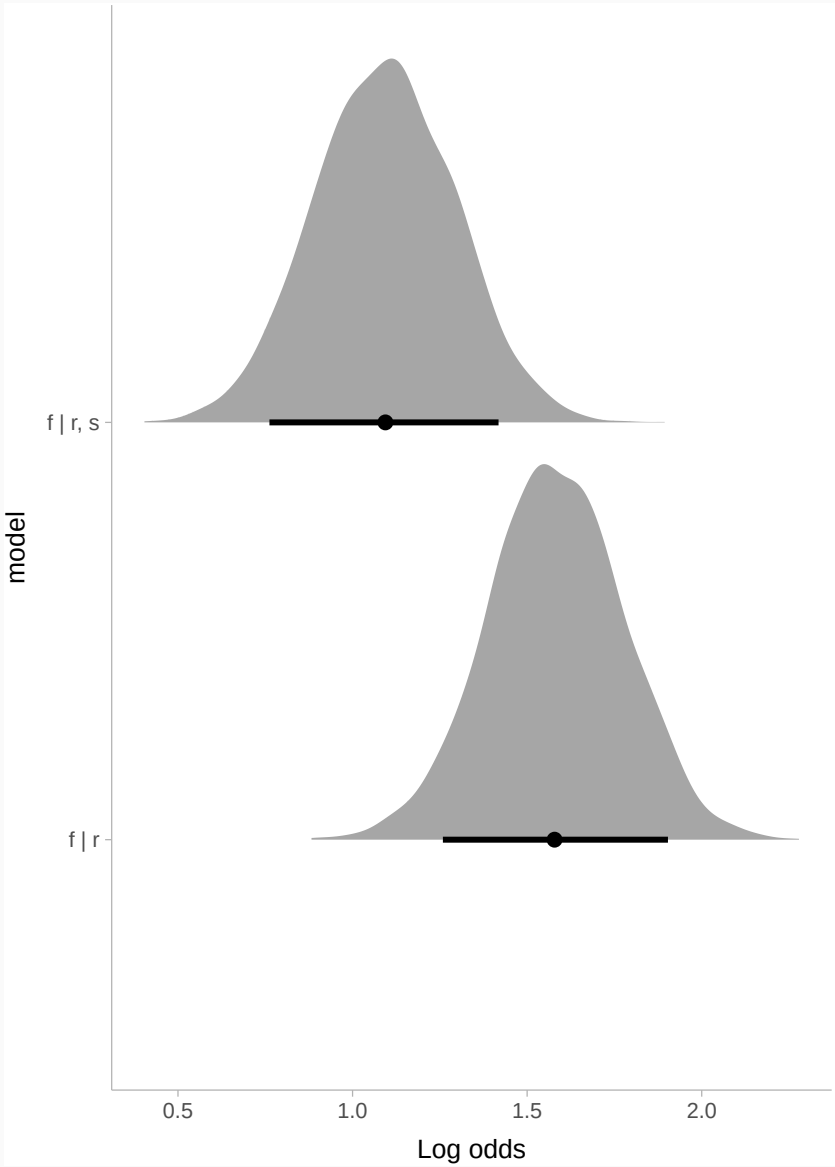
Estimate the model: admin data example

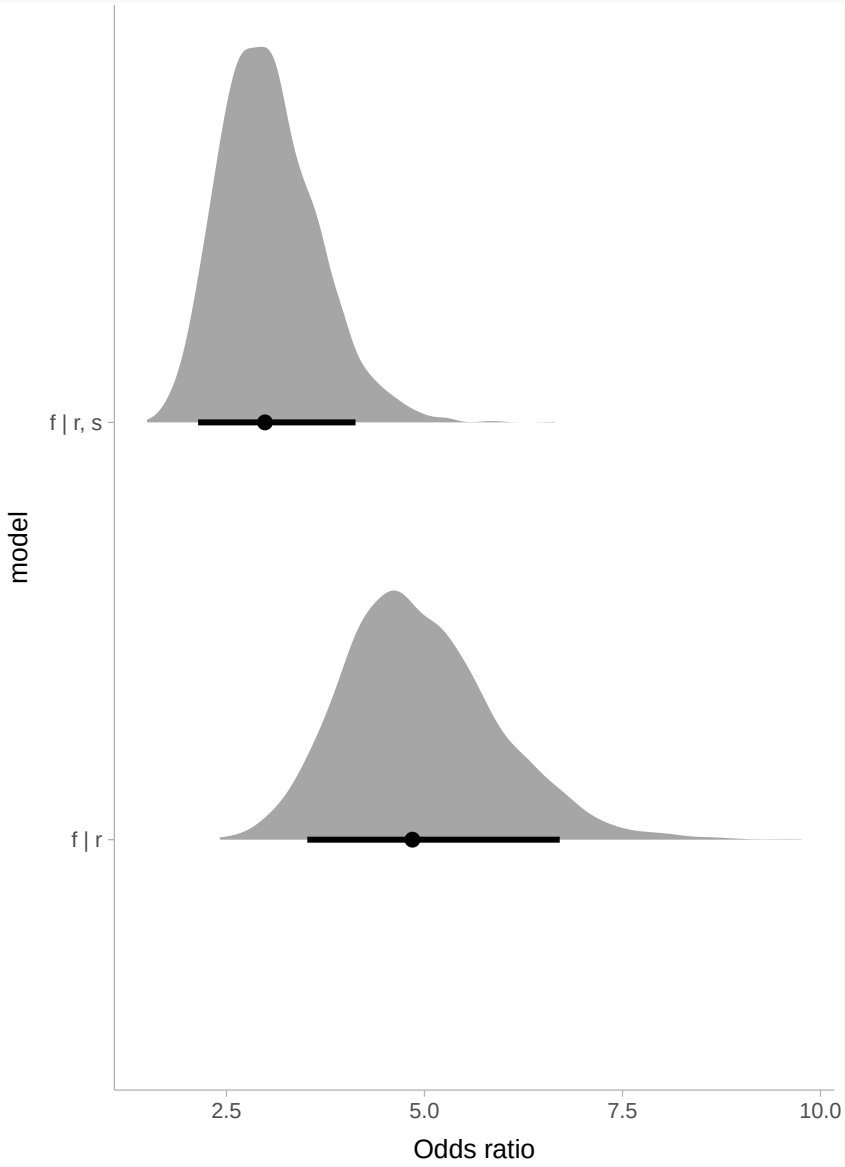
Here we'll only estimate the model for those who are stopped, in admin data we generally don't get information on those not stoped.

```
m_pol1<- brm(f~r, data = pol_sim_dat |>
               filter(s==T), family = "bernoulli")
```

Compare the posteriors: subset out those not stopped

What's going on here?

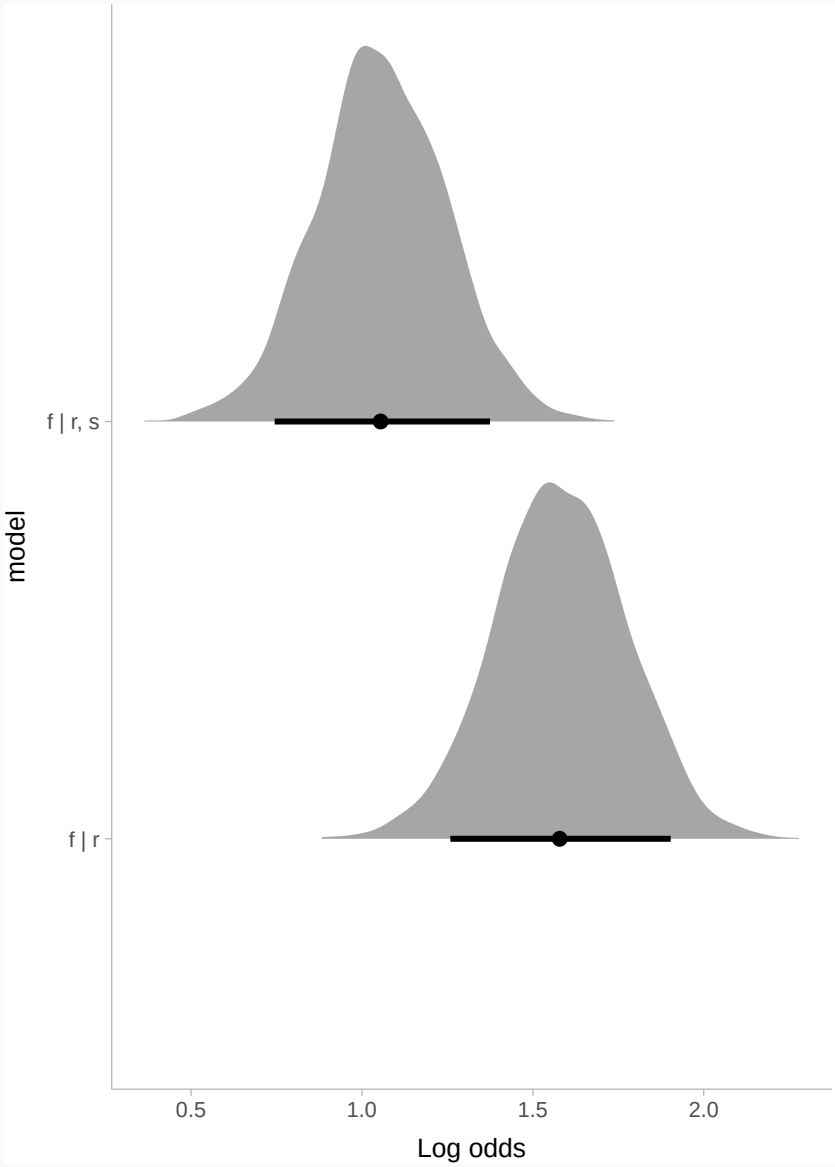


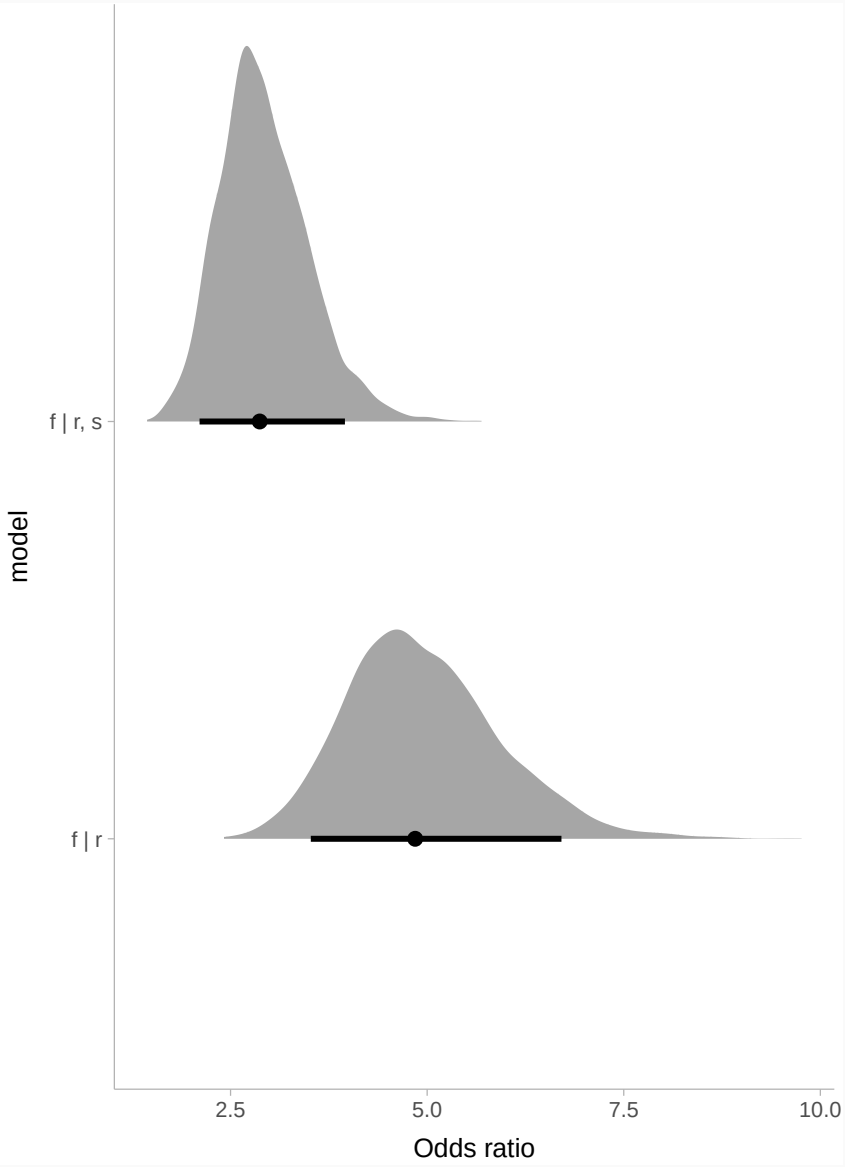


Estimate the model: ‘controlling for arrest’ example

Reviewer 2 insists: “You must control for rates of arrest! Otherwise you’ve got omitted variable bias!”

Compare the posteriors: stops as a control

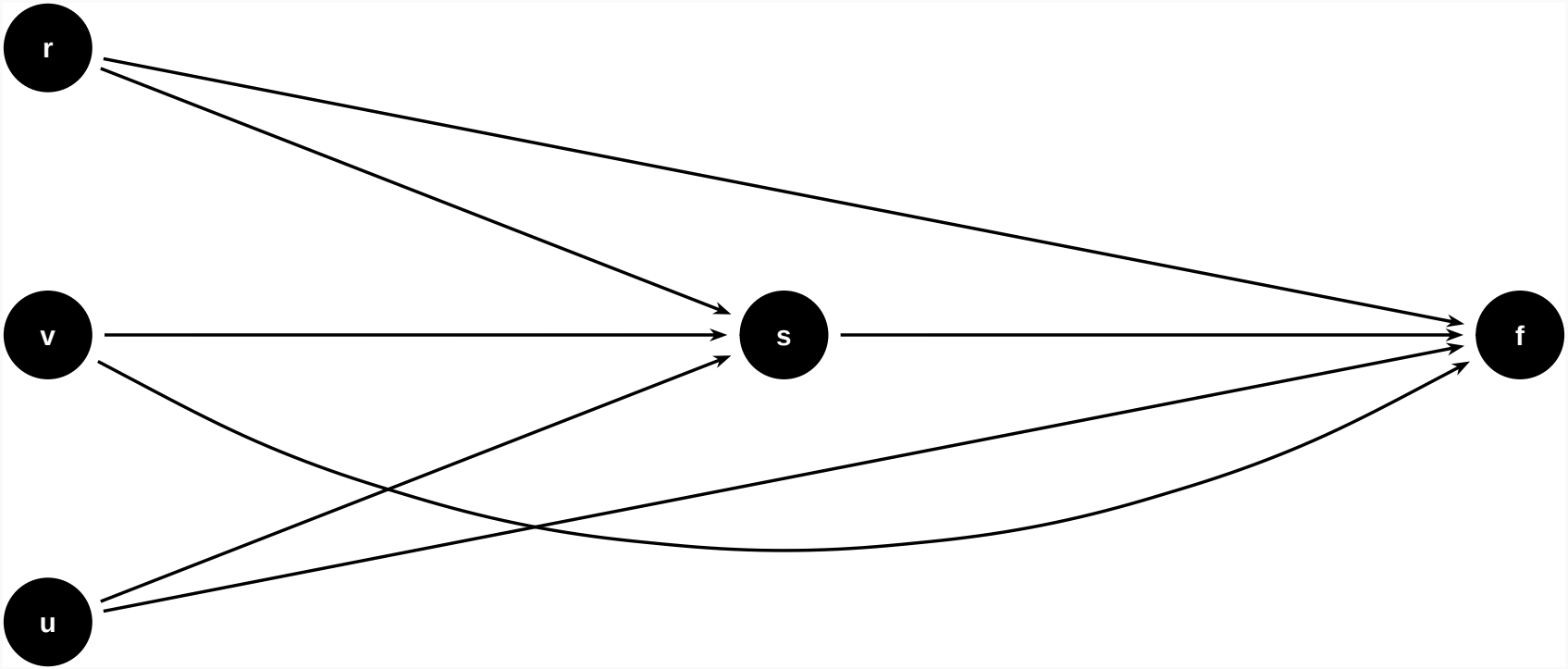




Making the example more realistic

Perhaps Reviewer 2 also says: "You have to control for whether the suspect was armed or violent! Include all the omitted variables!"

Let's assume that both black and white suspects who are violent are always stopped, but Black suspects who are not violent are stopped at higher rates than white suspects. This is our revised DAG.



Simulating violence

We'll now add the edges $v \to s$ and $v \to f$ to our simulation

Check the sims

Violent suspects far more likely to be stopped, as expected

r	v	p	s	f
FALSE	FALSE	80818	59.01	0.06
FALSE	TRUE	4203	449.44	9.75
TRUE	FALSE	14223	197.57	1.41
TRUE	TRUE	756	703.70	59.52

Conditioning on s

White suspects who are stopped are more violent on average under this scenario. They have a higher bar for police making a stop.

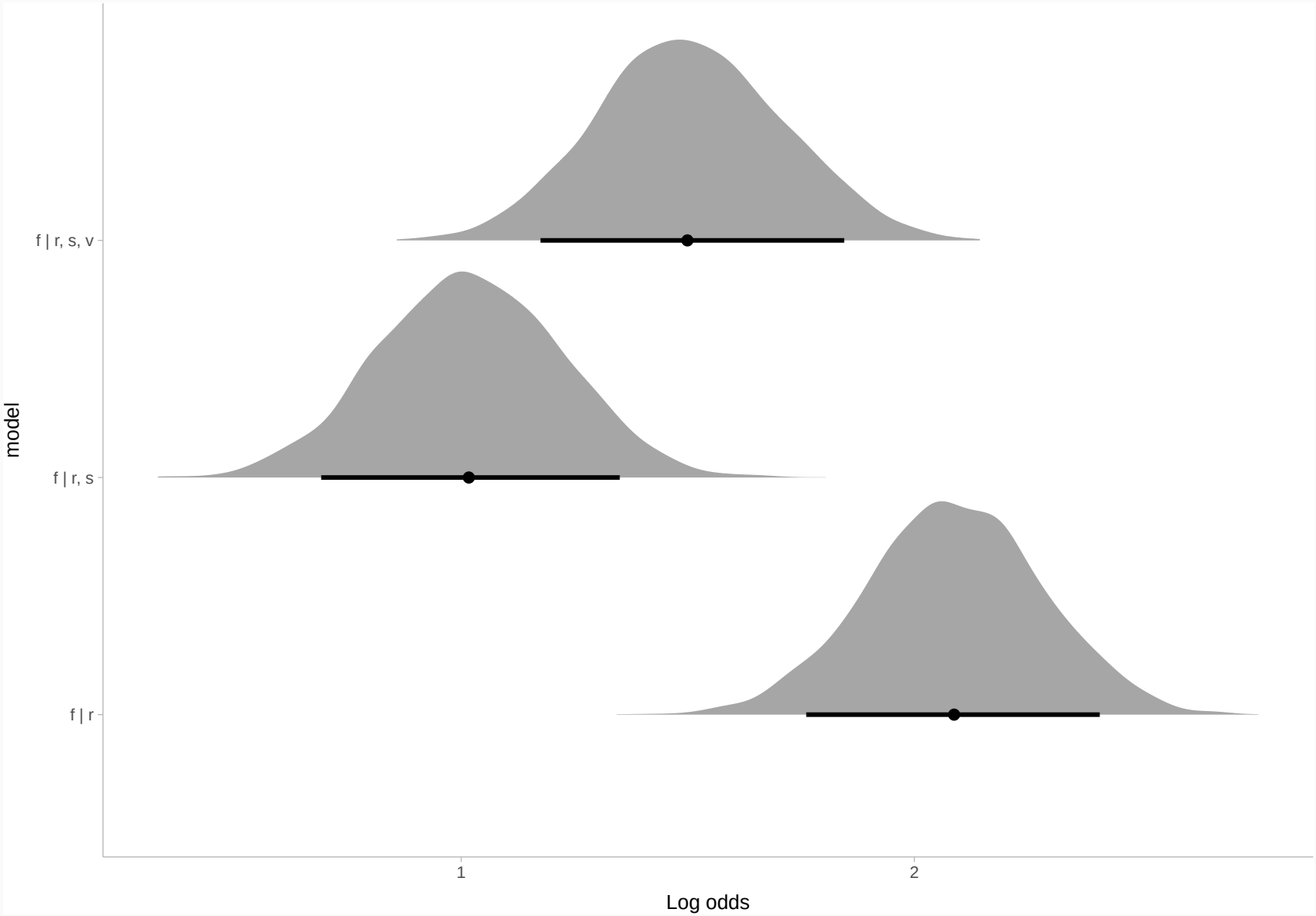
```
pol_sim_dat |>
  filter(s==1) |>
  group_by(r) |>
  summarize(v = mean(v)) |>
  knitr::kable(digits = 2)
```

r	v
FALSE	0.28
TRUE	0.16

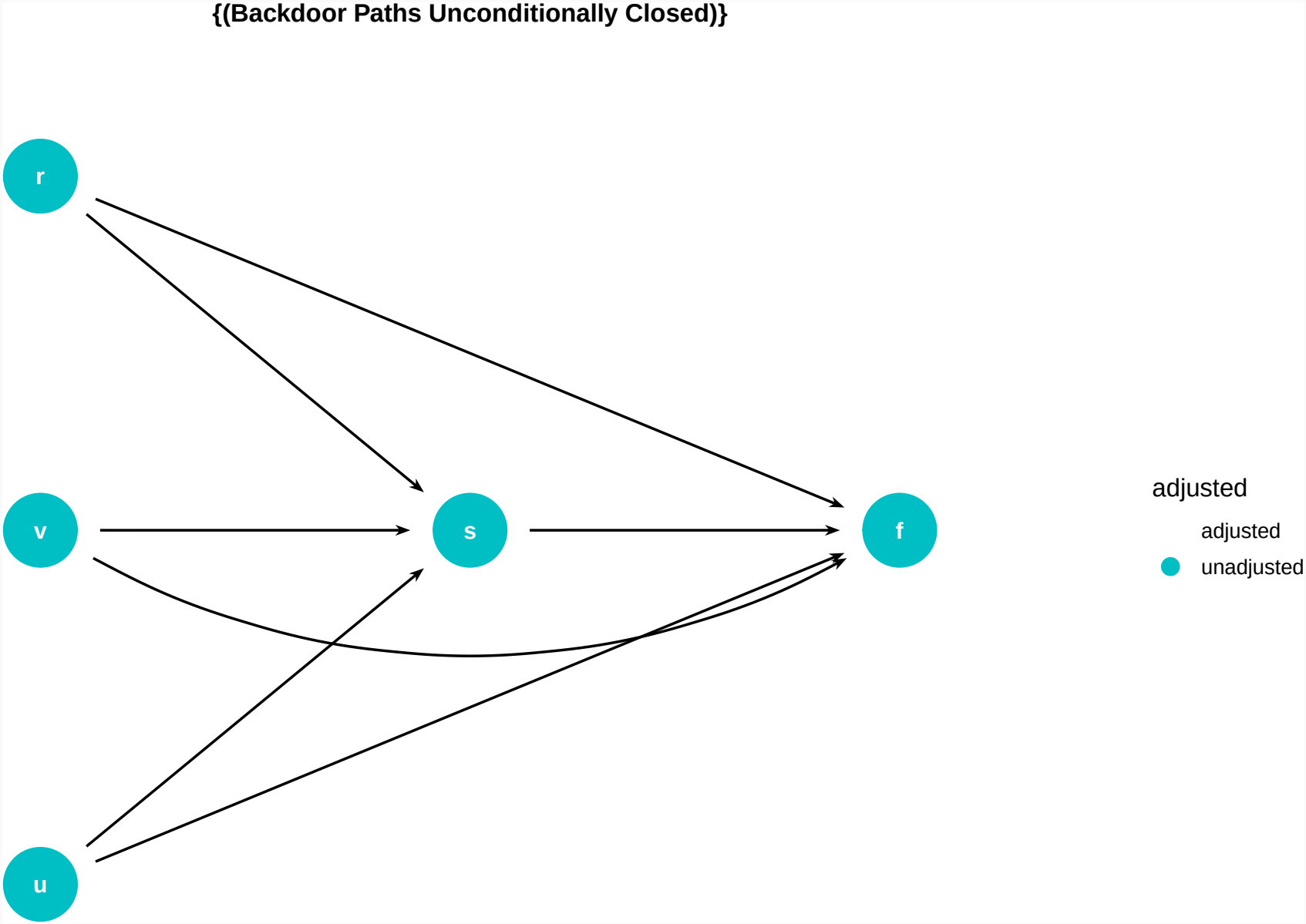
Fitting the models

```
m_pol0 <- brm(f ~ r, data = pol_sim_dat, family = "bernoulli", cores=4)
m_pol1 <- brm(f ~ r, data = pol_sim_dat |>
                filter(s==T), family = "bernoulli", cores=4)
m_pol2<- brm(f ~ r + v, data = pol_sim_dat |>
               filter(s==T), family = "bernoulli", cores=4)
```

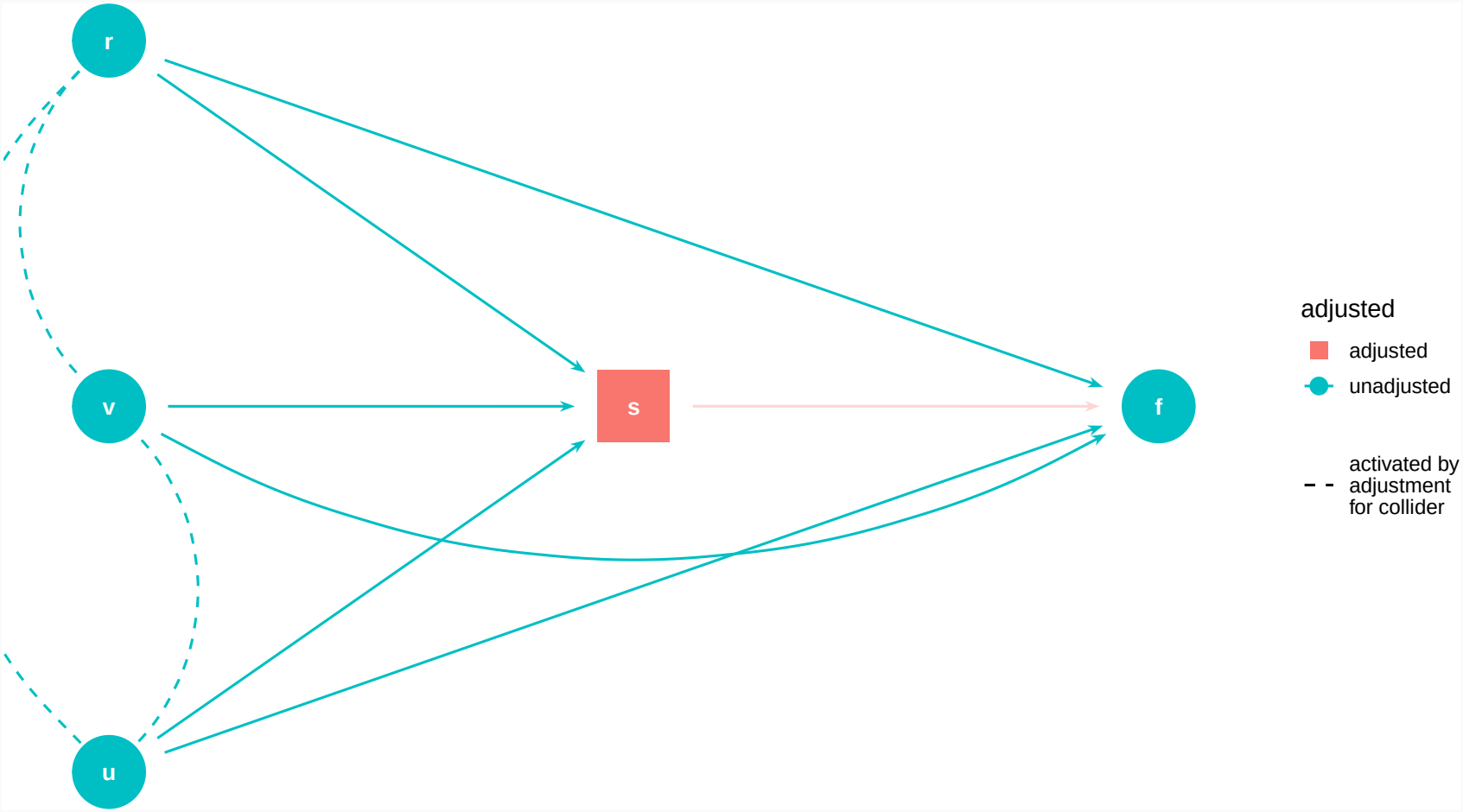
Checking the posteriors



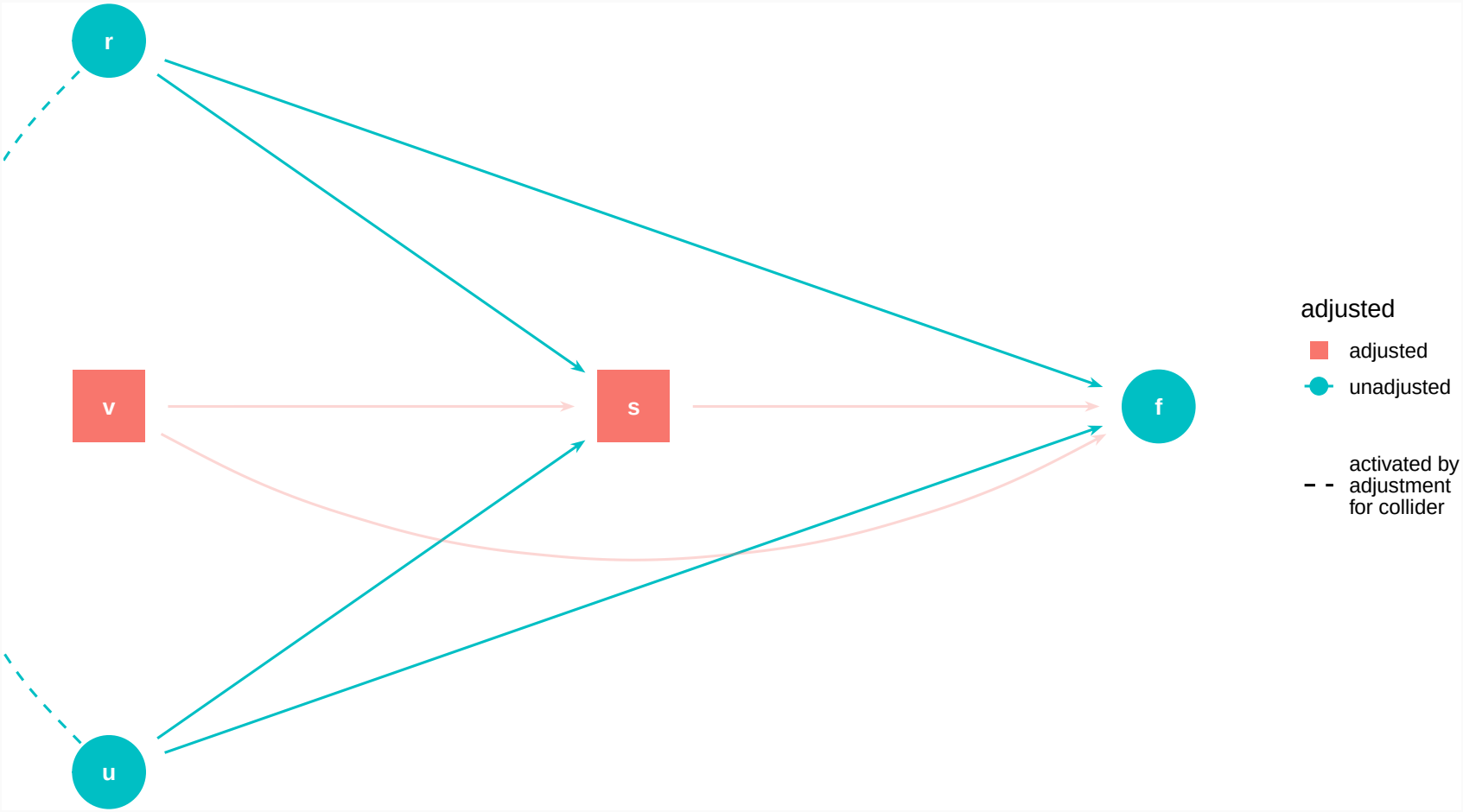
So which is correct?



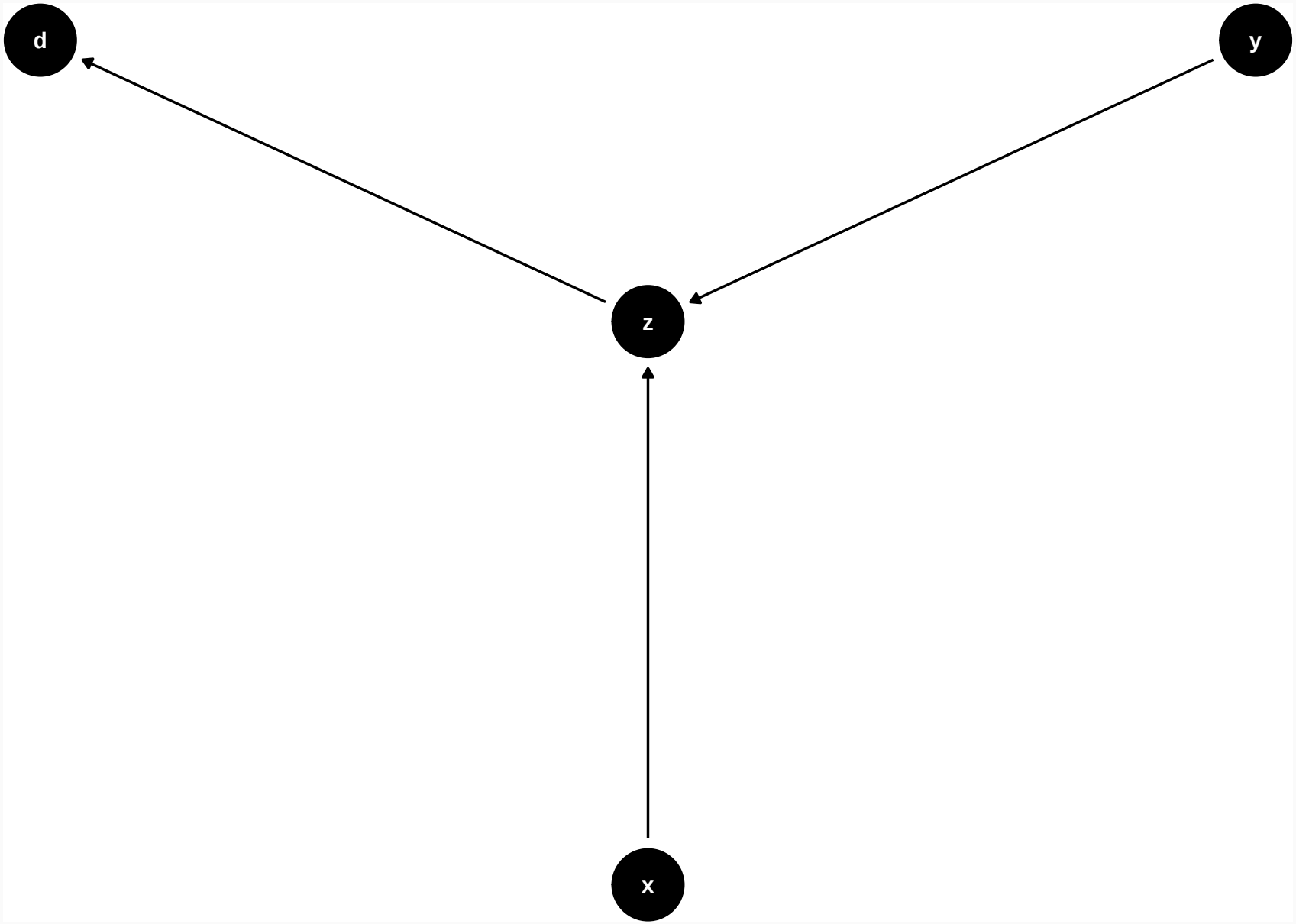
So which is correct?



So which is correct?



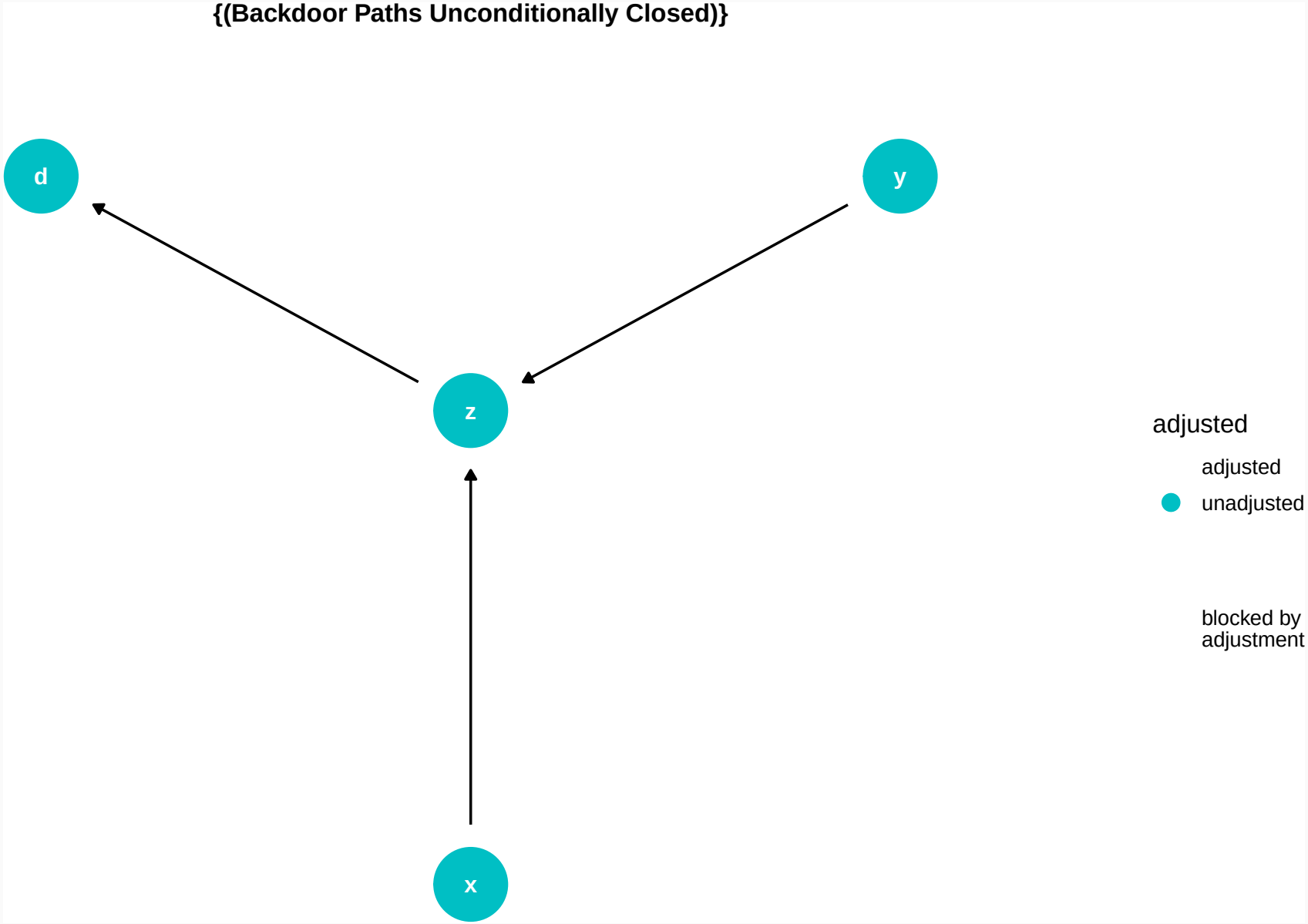
The descendant



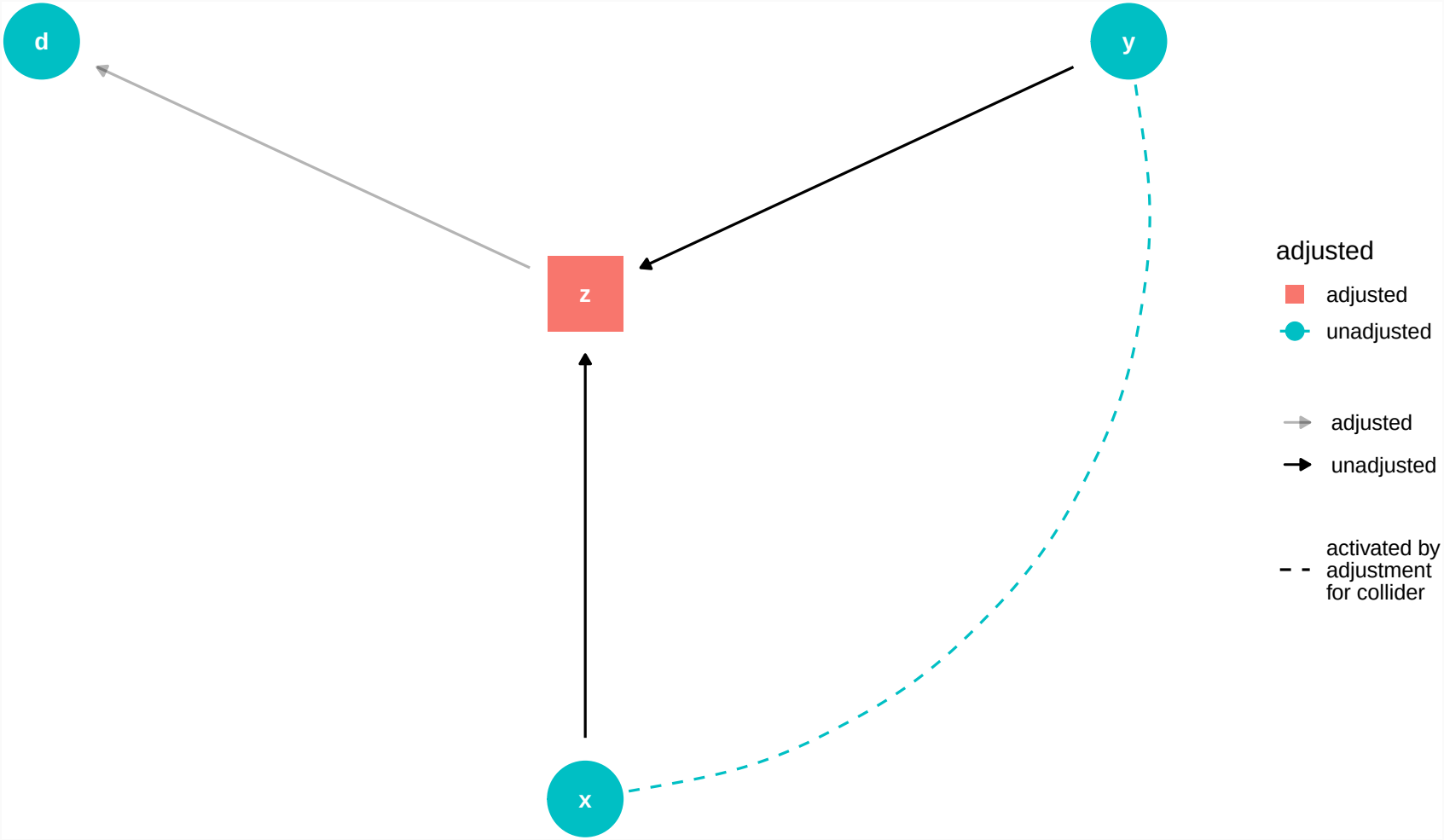
Basic features

- d is a downstream consequence of z
- variables like d are often used as *proxies* for z
- the consequences of adjustment depend on relations between ancestor nodes

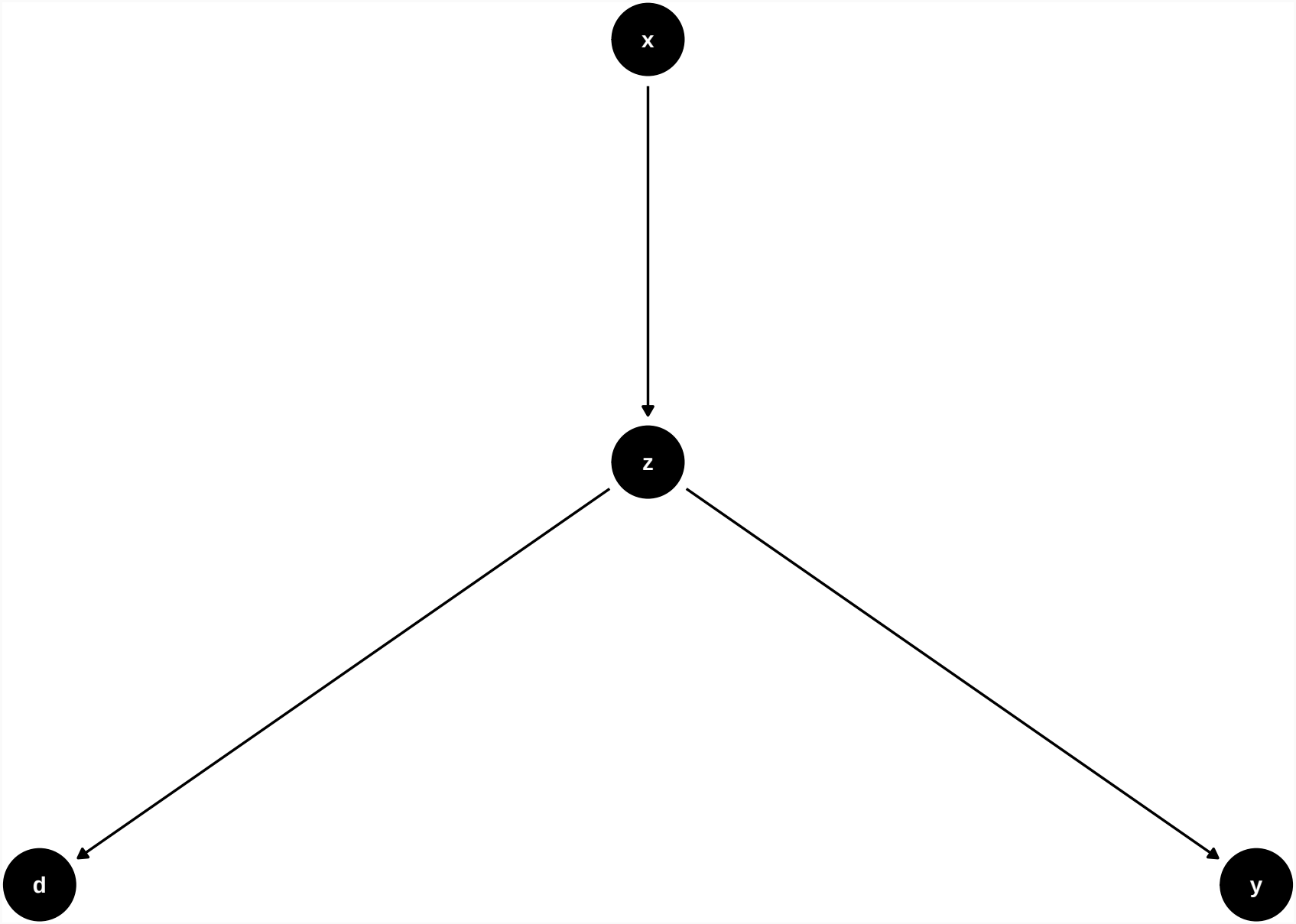
Identifying the effect of x on d



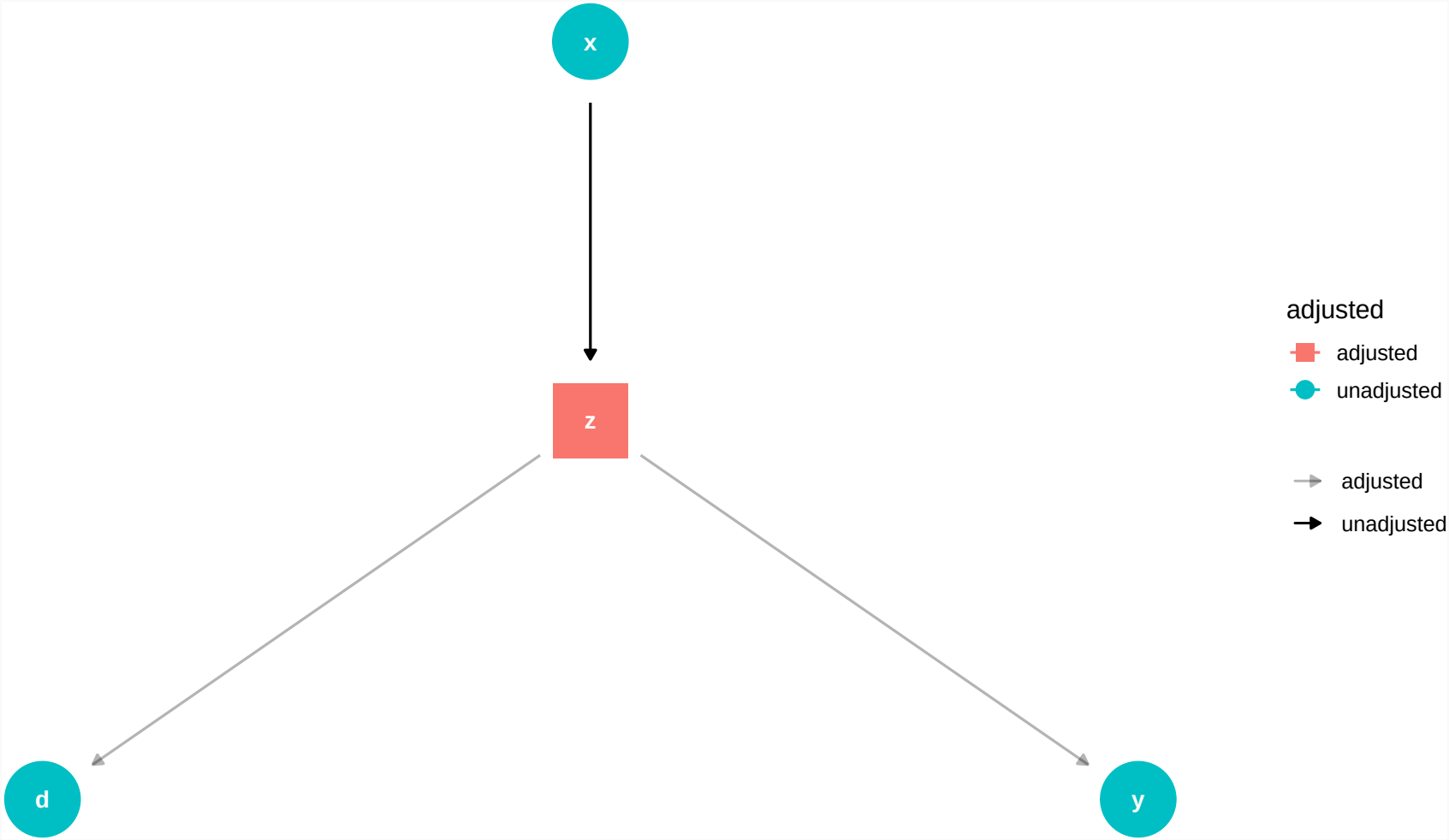
A bad control



Under a different structure

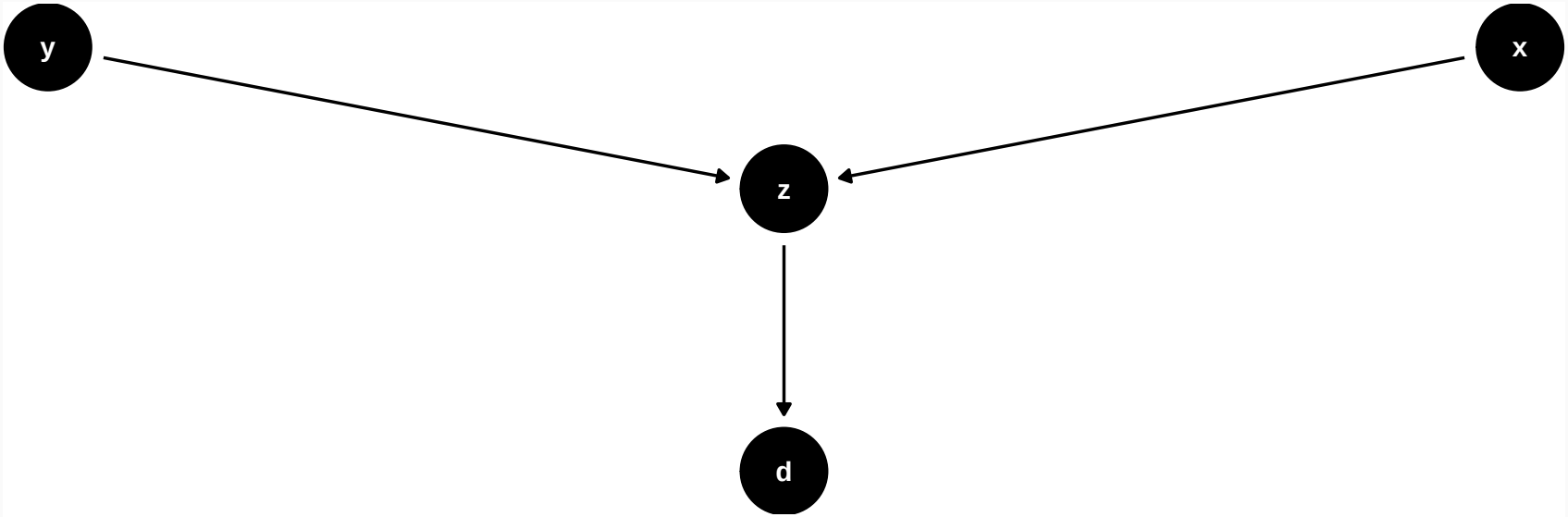


A control that is also bad but for another reason



Measurement by proxy, a simulation

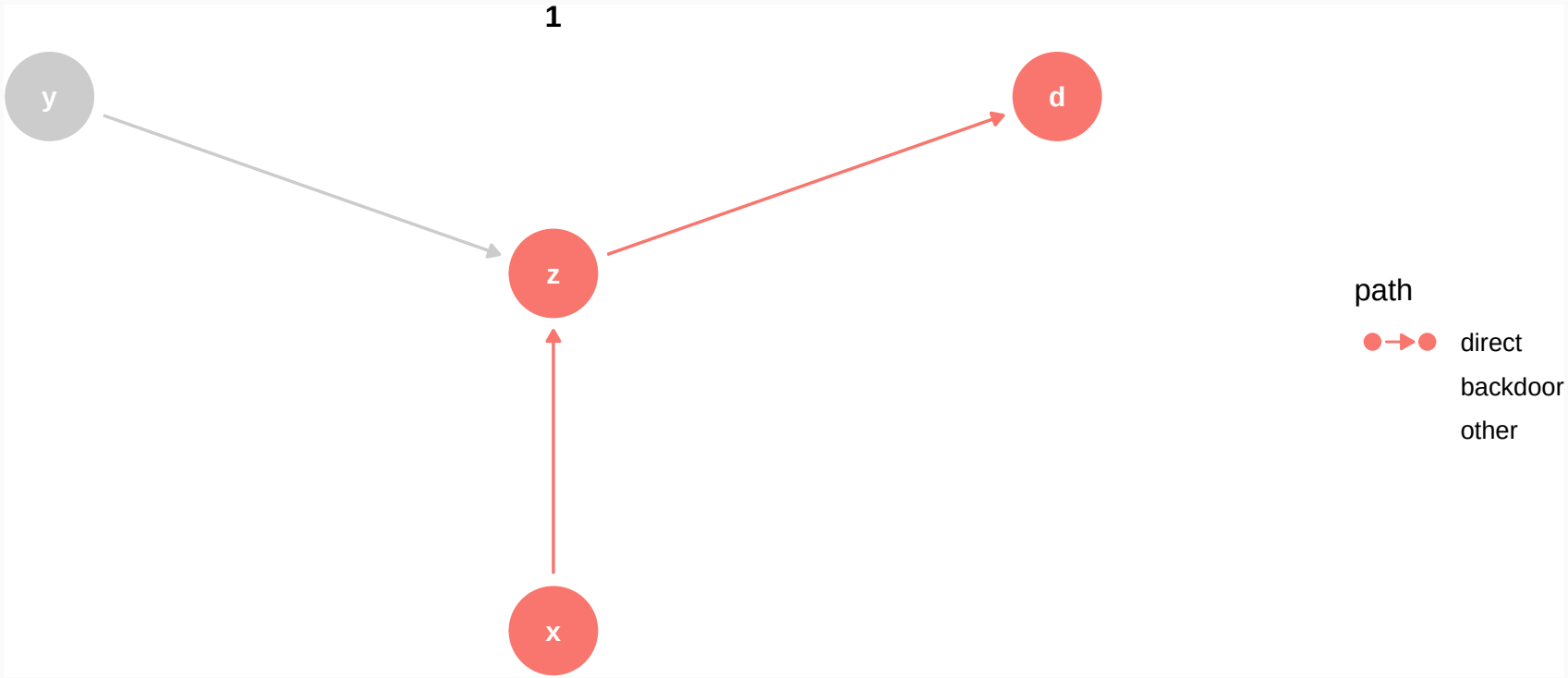
Let's assume this dag. We want to measure the effect of x on d.




```
# simulate descendant
n_sims <- 1000
x <- rnorm(n_sims, 0, 1)
y <- rnorm(n_sims, 0, 1)
z <- rnorm(n_sims, x+y, 1)
d <- rnorm(n_sims, z, 1)
simdat <- data.frame(x, y, z, d)
```

Let's look at the paths

To estimate the effect $x \longrightarrow d$, we should *not* condition on z

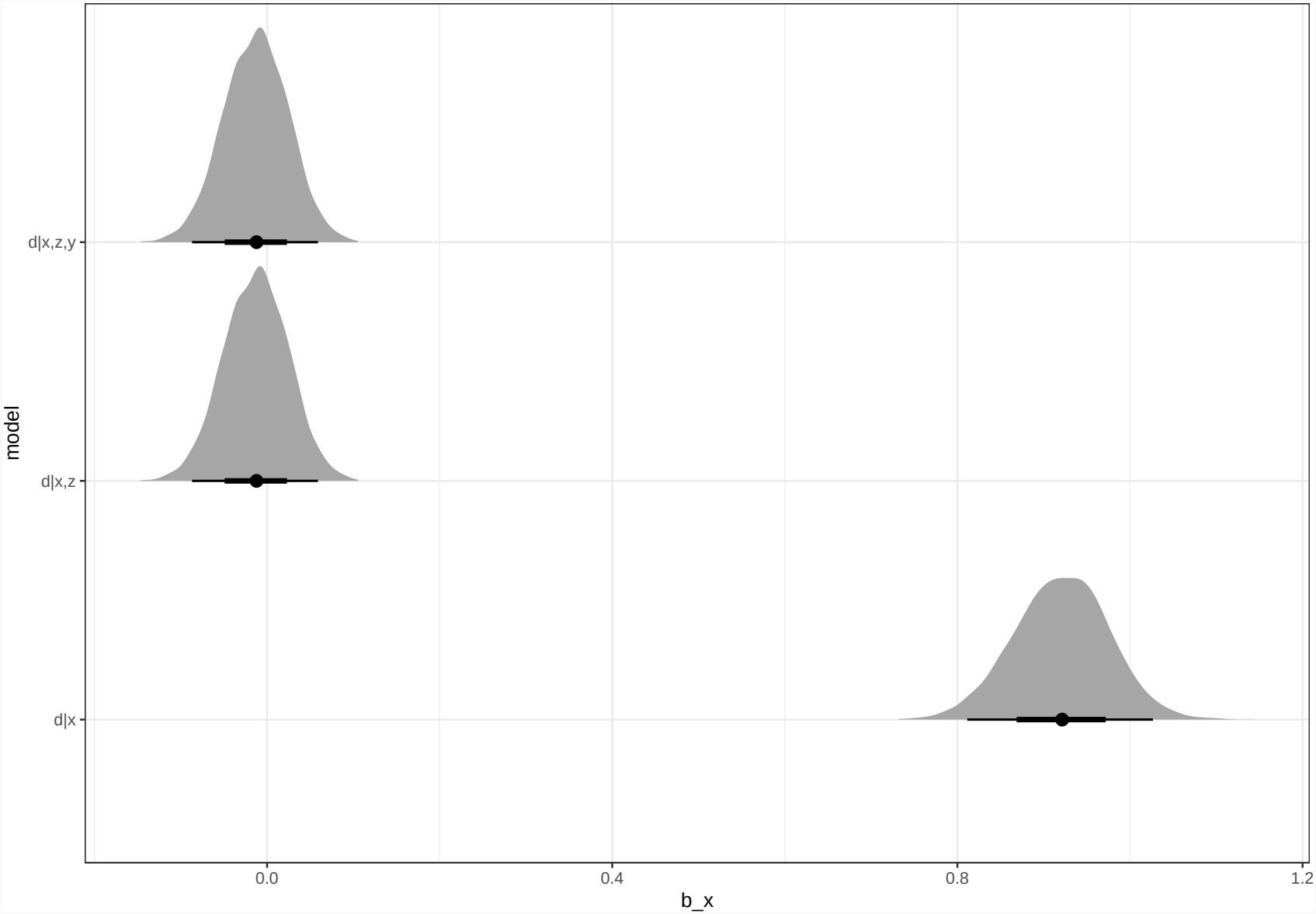


Estimation

```r
# correct model, no conditioning
m_d0 <- brm(d~x, data = simdat)
# adjusting for z, reviewer 2 told you to
m_d1 <- brm(d~x + z, data = simdat)
# adjusting for z and y, reviewer 3 told you to
m_d2 <- brm(d~x + z + y, data = simdat)

## extract posteriors
postd0 <- as_draws_df(m_d0) |> mutate(model = "d|x")
postd1 <- as_draws_df(m_d1) |> mutate(model = "d|x,z")
postd2 <- as_draws_df(m_d1) |> mutate(model = "d|x,z,y")
```

Posterior visualization

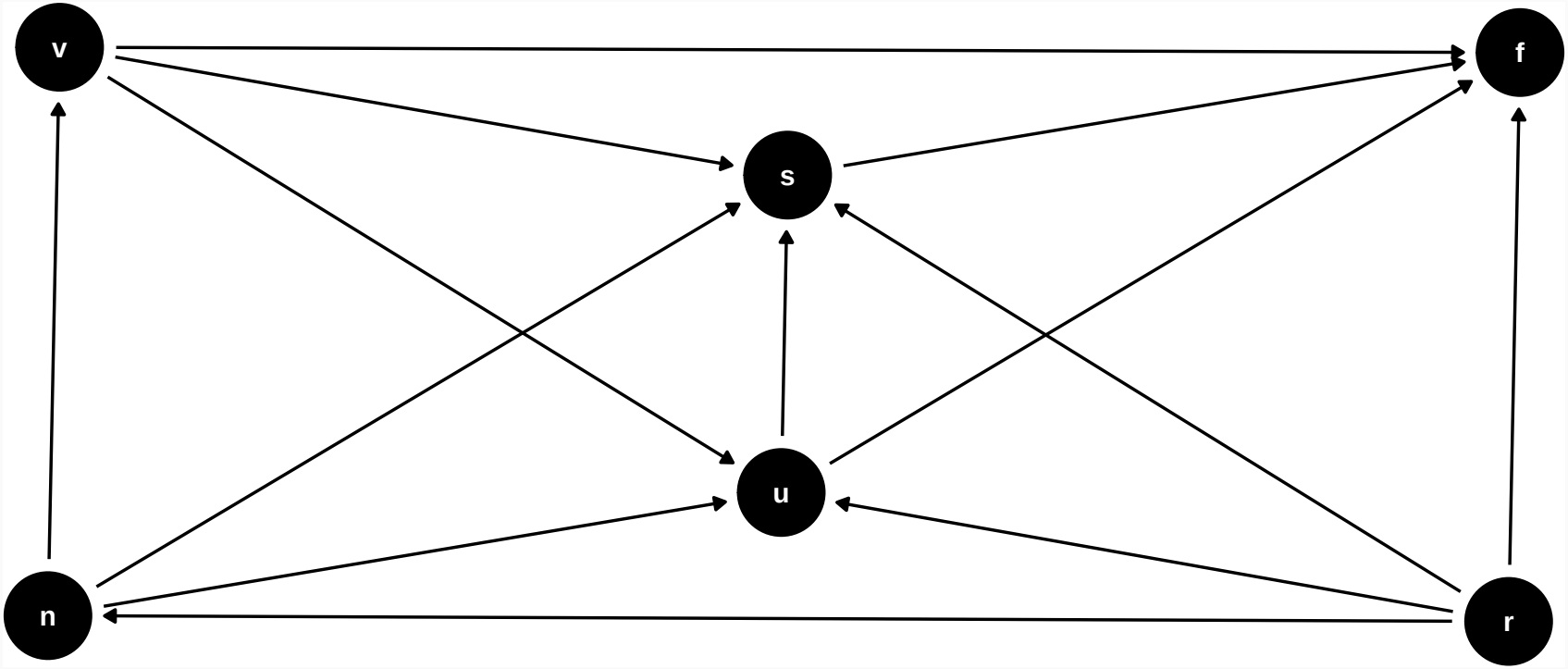


The workflow

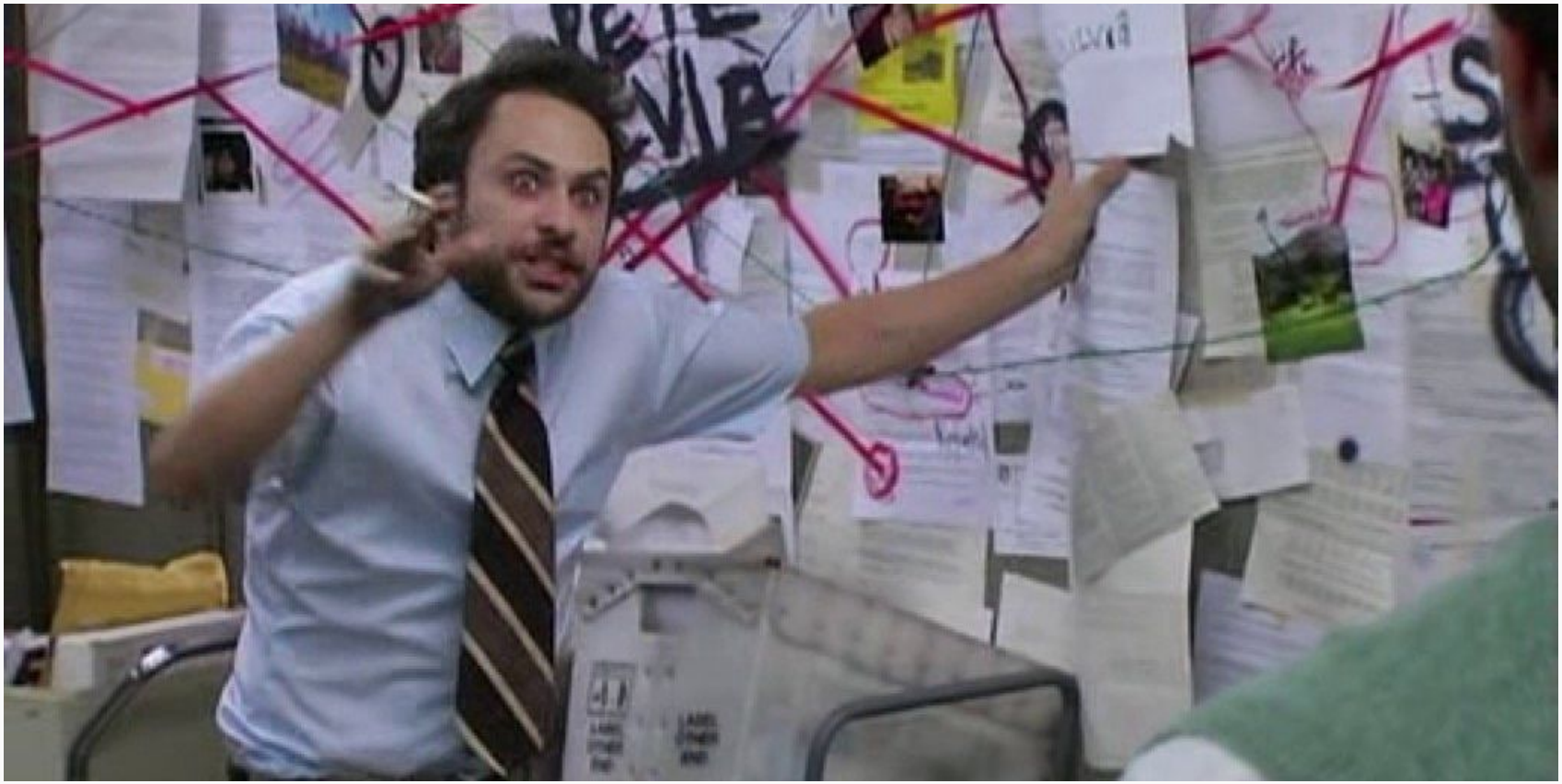
1. Specify your DAG.
2. List all of the paths connecting a cause of interest x to an outcome y.
3. Classify all paths as open or closed. A path is open unless it contains a collider.
4. Identify all backdoor paths.
5. If needed, choose variables to condition on to close backdoor paths.

A more realistic example

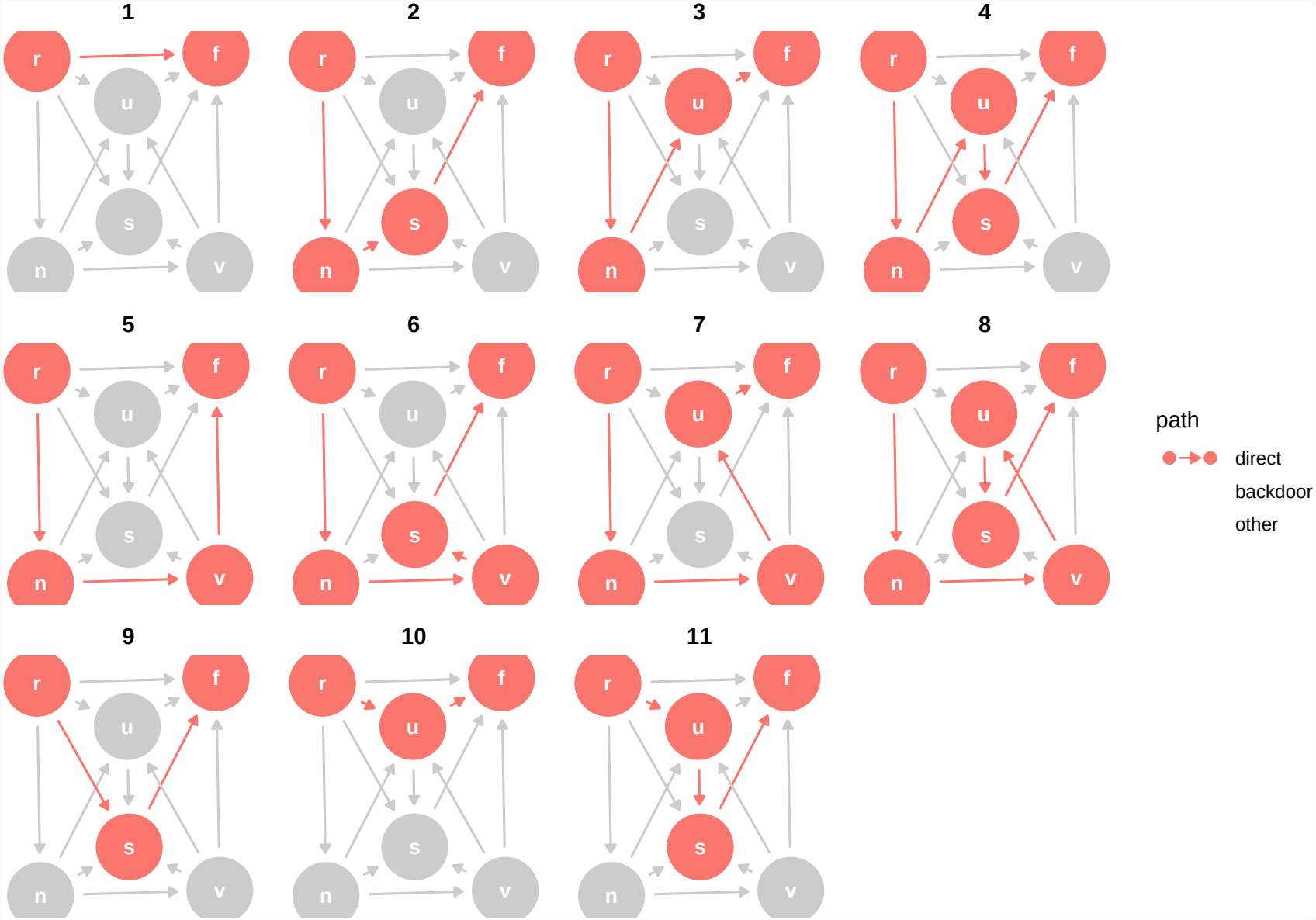
What paths are open between r and f?



What I’ve been told I look like when I draw these



Identifying the paths



Consequences of adjustment

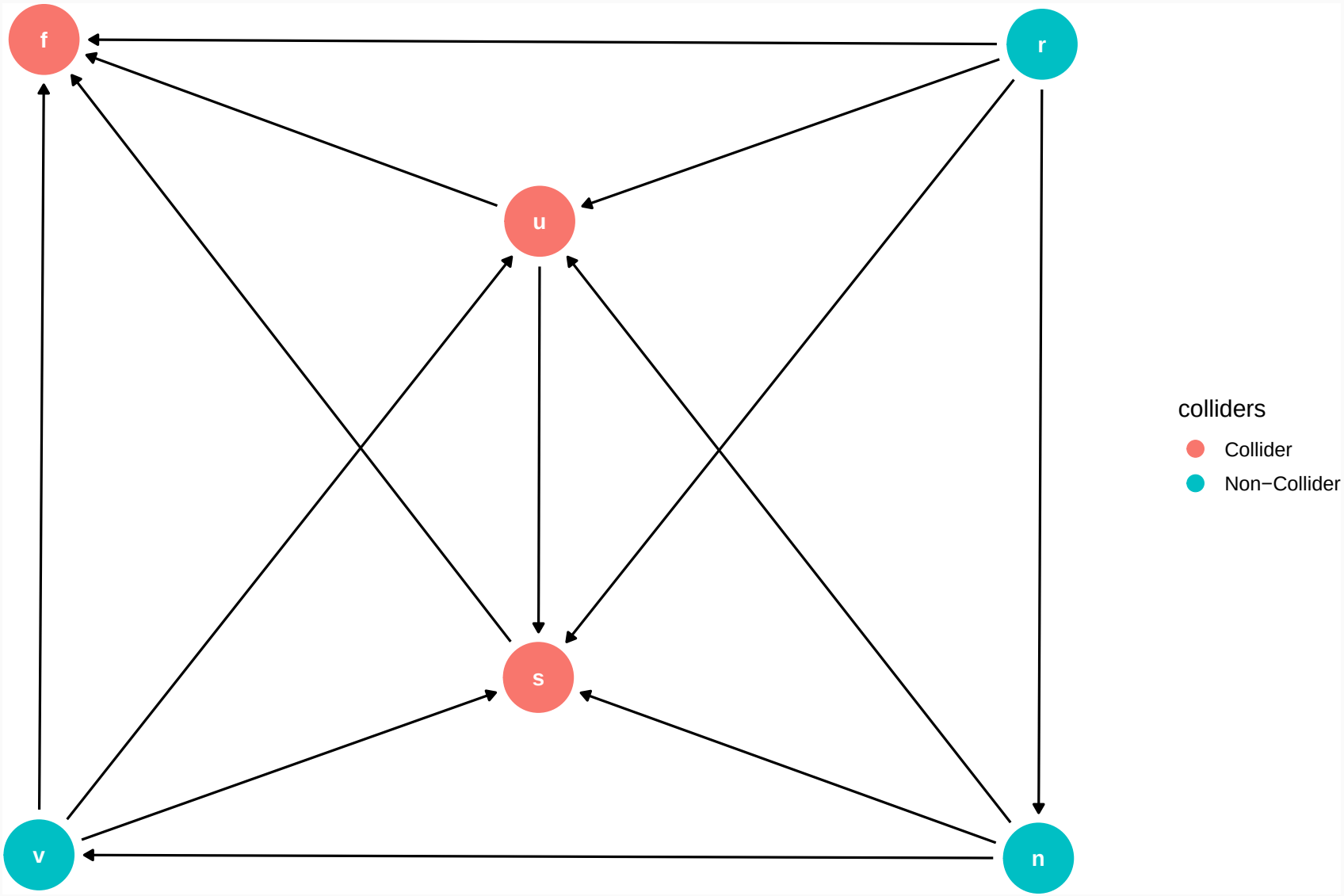
The DAG suggests we shouldn't adjust anything to obtain p(f|r)

```
adjustmentSets(poldag3)
```

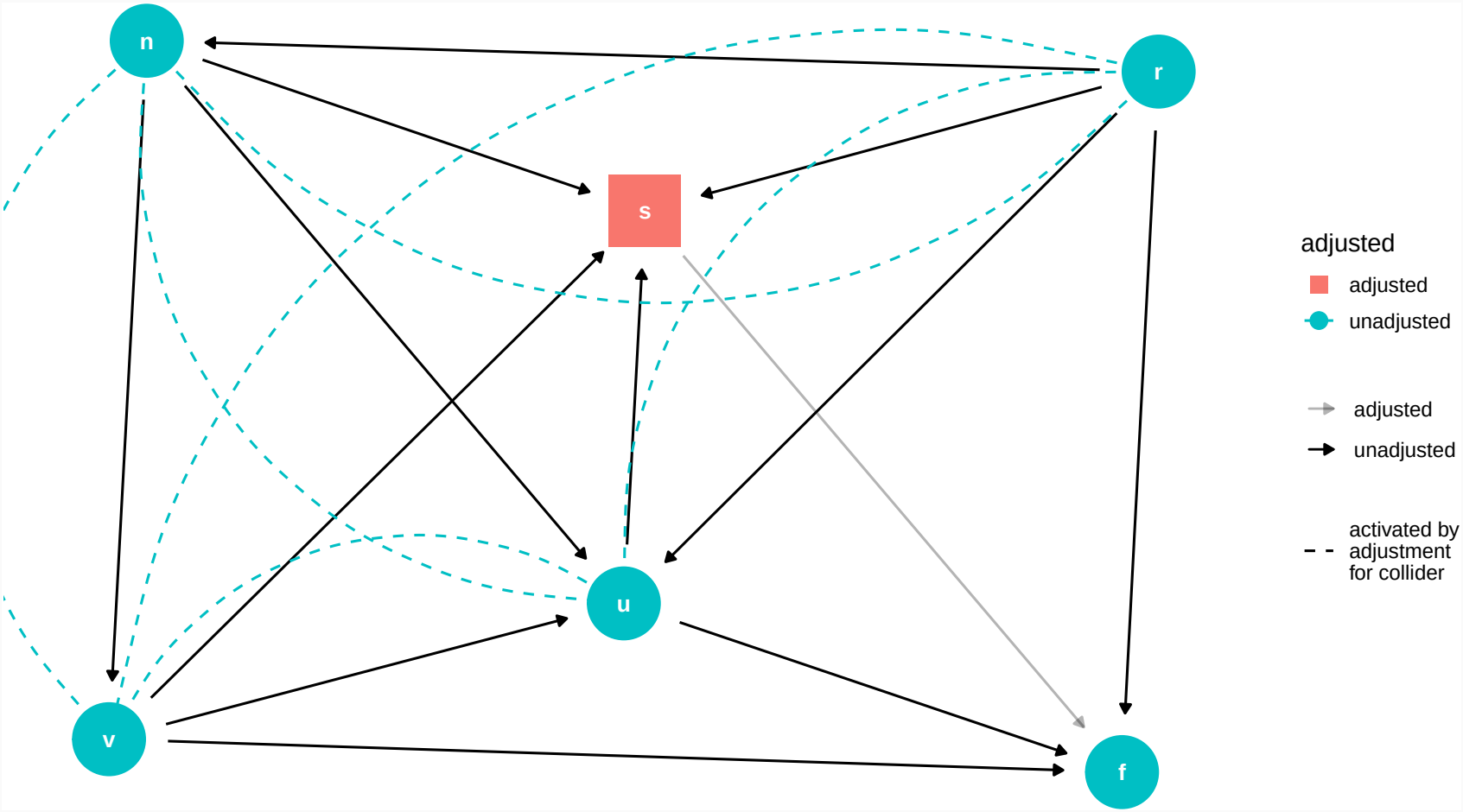
```
{}
```

But reviewer 2 says include all the controls!

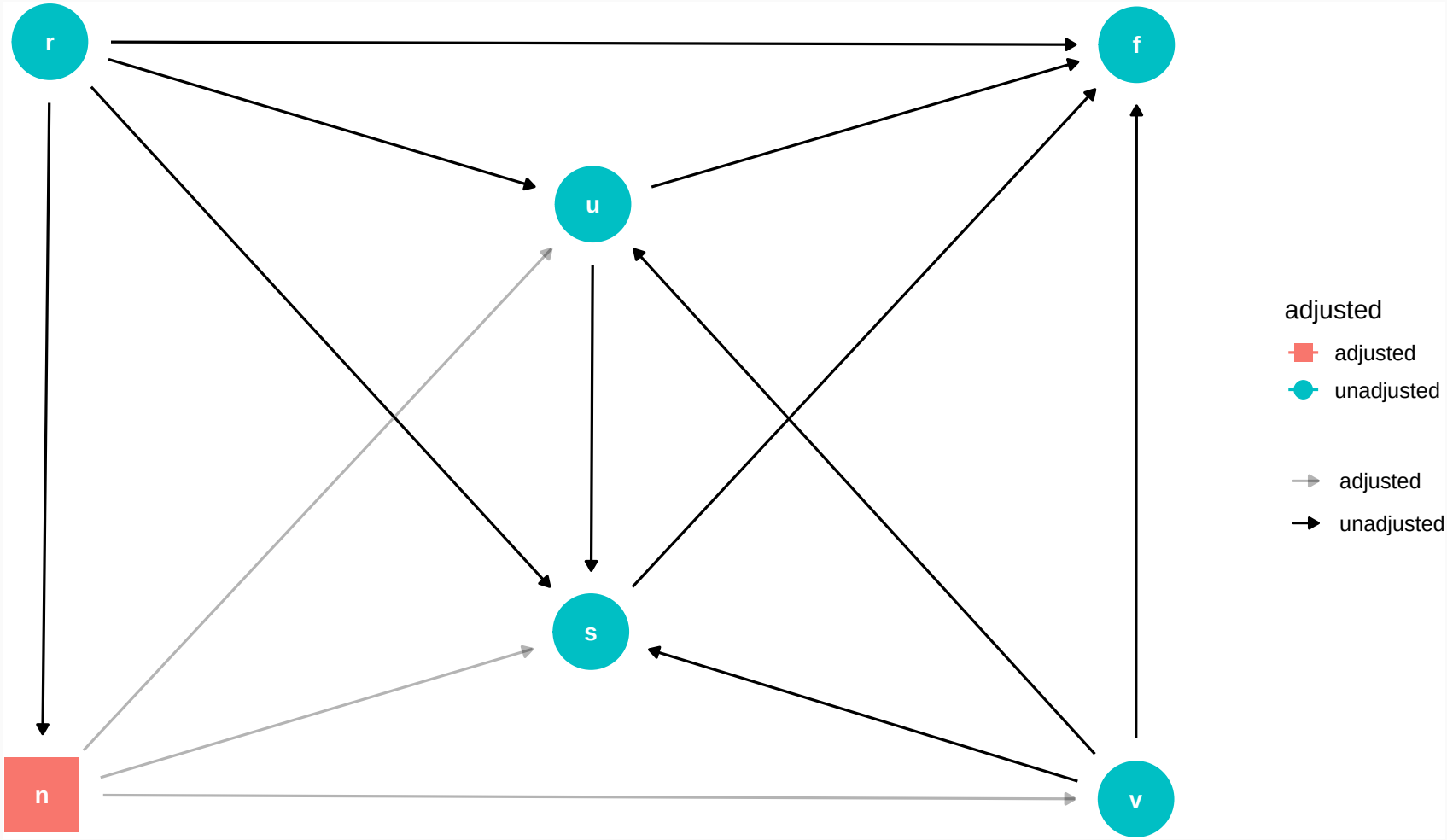
Bad controls: colliders



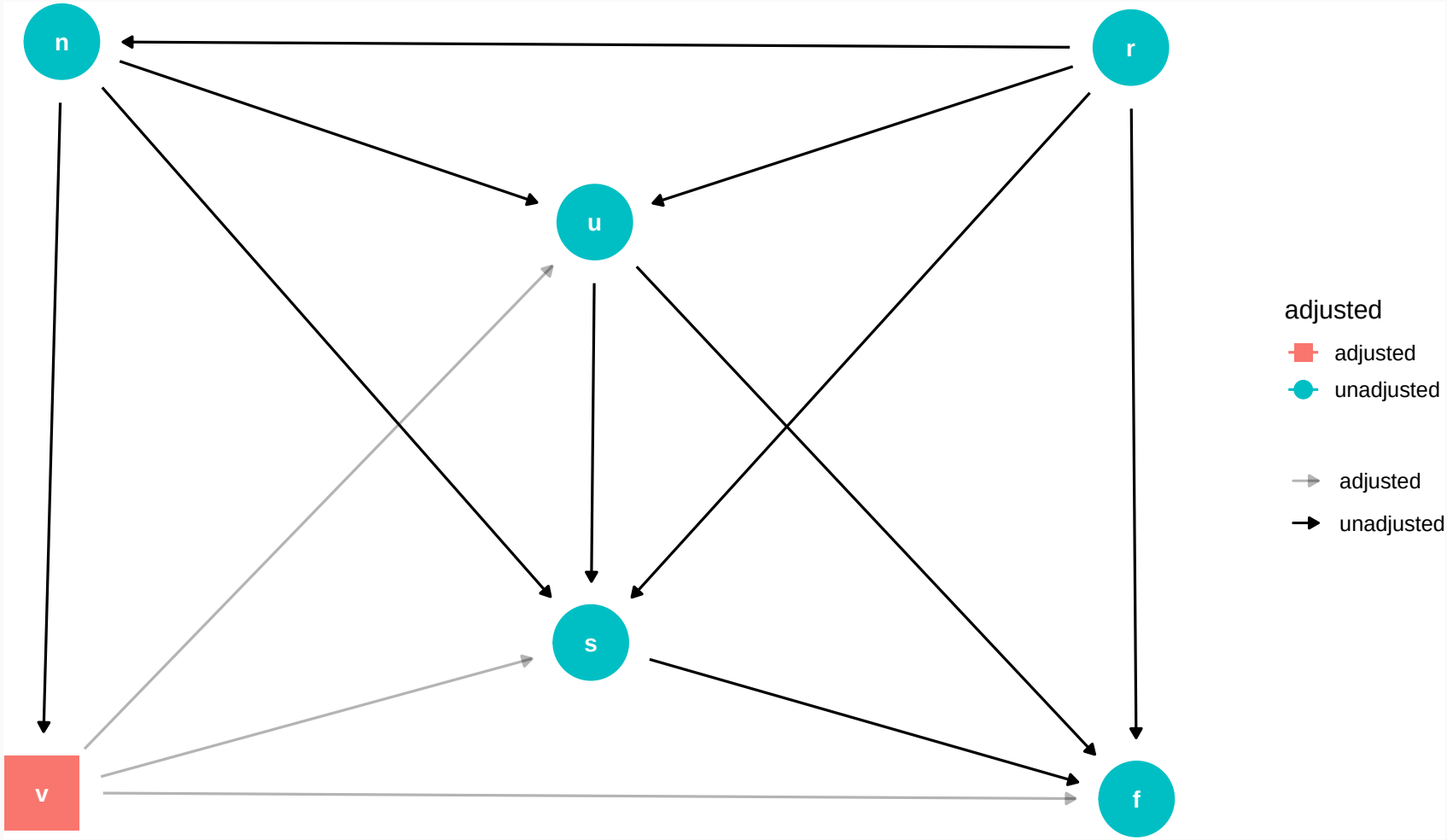
Bad controls: colliders



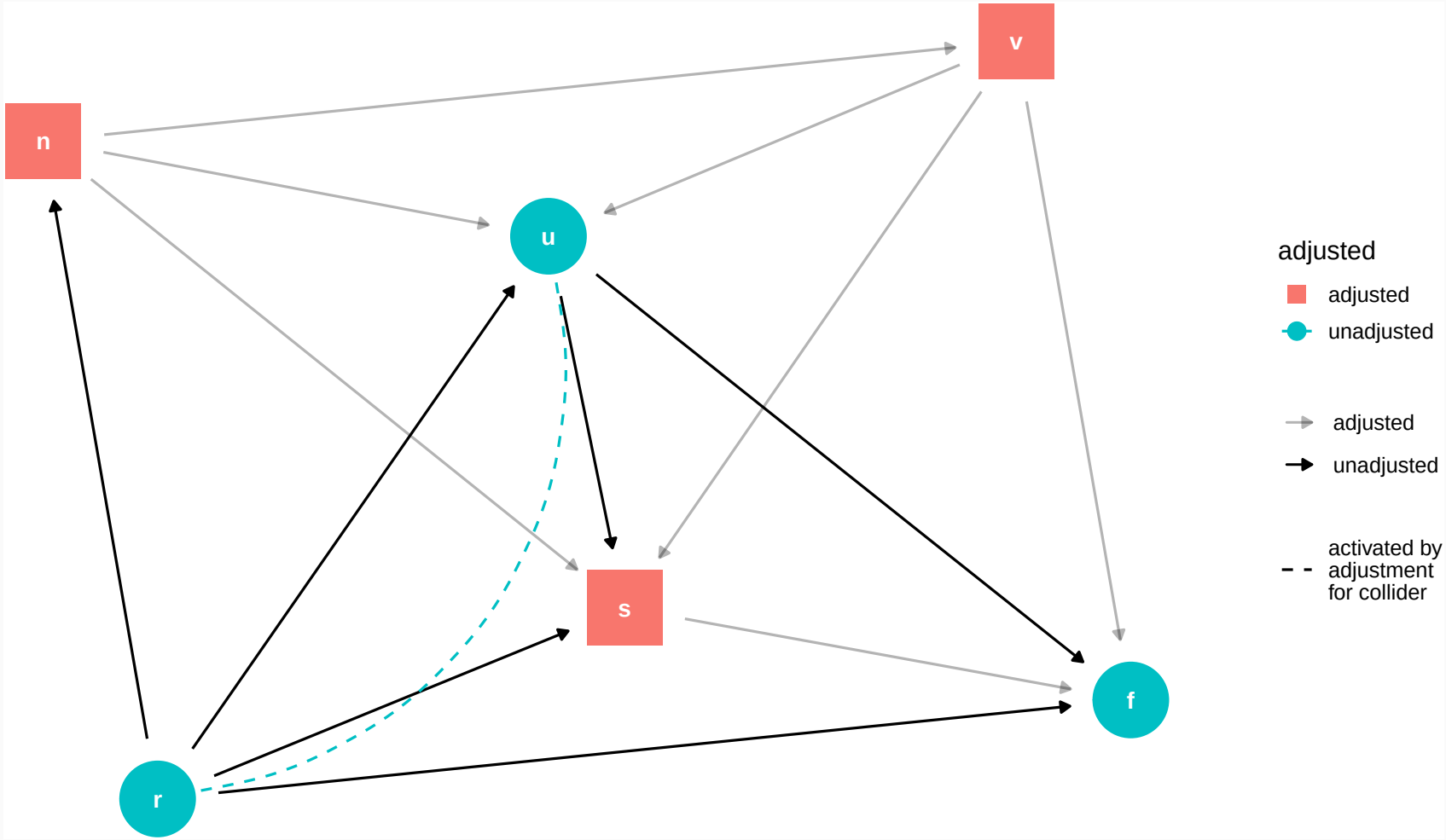
Bad controls: path blockers



Bad controls: path blockers



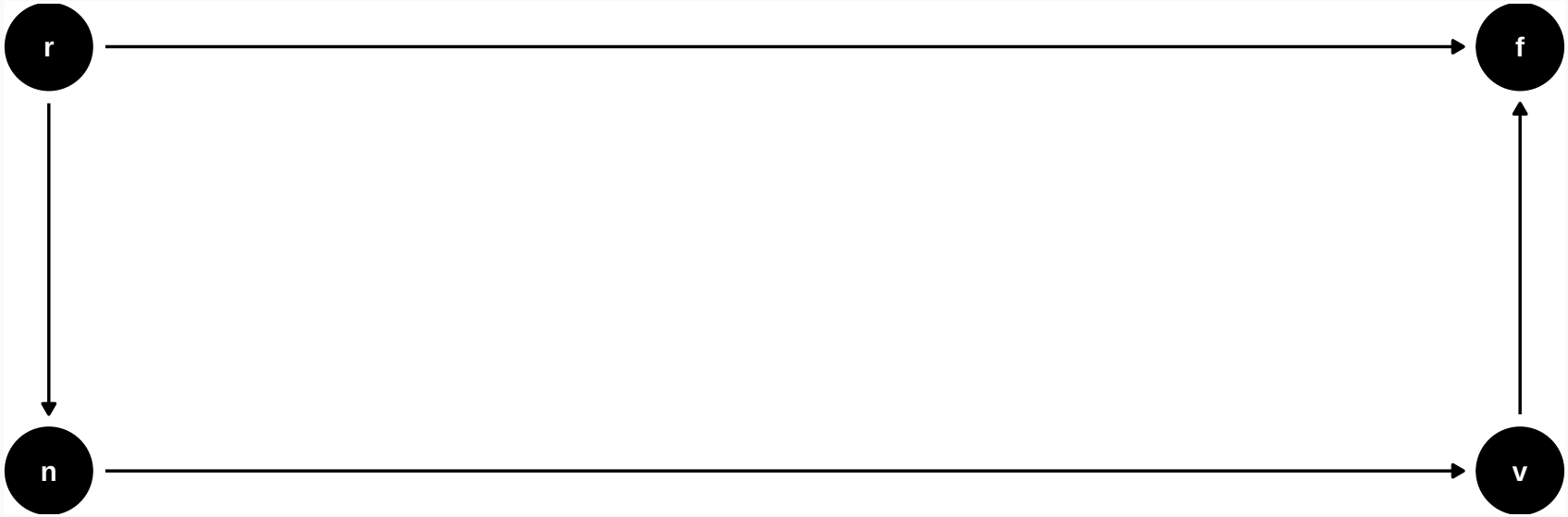
Estimating a ‘direct’ effect



Imagining an intervention: what would happen if we intervene on v?

The ‘do’ operator helps us imagine the effects of interventions. We use it to delete arrows into a node. $p(f|r, do(v))$. Imagine we intervene on violence through community interventions, how would this affect the relationship between police violence and race?

Let’s assume this simple DAG



do(v)

We can use the do operator to signal intervention, $p(f|r, do(v))$ deleting arrows into v. Now the effect $r \rightarrow f$ does not pass through $n \rightarrow v$. We can generate this effect on the DAG by conditioning on v.

